Lecture 12. Hardness in P

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

I've got data. I want to solve this algorithmic problem but I'm stuck!



Ok, thanks, I feel better that none of my attempts worked. I'll use some heuristics.

I'm sorry, this problem is NP-hard. A fast algorithm for it would resolve a hard problem in CS/math.



I've got data. I want to solve this algorithmic problem but I'm stuck!



But my data size n is huge! Don't you have a faster algorithm?

?!?... Should I wait? or should I be satisfied with heuristics?

Great news! Your problem is in P. There is an $O(n^3)$ time algorithm

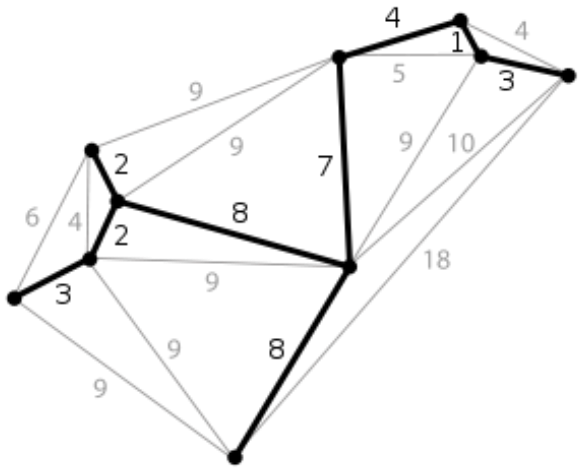


Uhm, I don't know… This is already theoretically fast… For some reason I can't come up with a faster algorithm for it right now…

Research on problems in P

- Some problems have more and more progress…

Minimum
Spanning tree



O(m·log n)

O(m·loglog n) [Yao 1974]

O(m·log*n)
[Fredman, Tarjan 1987]

Randomized O(m)
[Karger, Klein & Tarjan 1995]

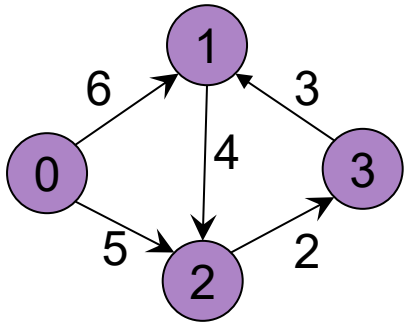
proved OPTIMAL
[Pettie and Ramachandran 2002]

O(m·α(m,n))
[Chazelle 1999]

deterministic O(m) ?

All pair shortest path (APSP)

- Given a real-weighted directed graph, find the distance between every two vertices



	0	1	2	3
0	0	6	5	7
1	∞	0	4	6
2	∞	5	0	2
3	∞	3	7	0



The Floyd-Warshall algorithm

- Running time $O(n^3)$

Initially,

distance[u][u]=0,	for all u∈V
distance[u][v]=w(u,v)	for all edge (u,v)∈E
distance[u][v]=+∞	otherwise

```
for k from 1 to |V|
   for i from 1 to |V|
      for j from 1 to |V|
         if distance[i][j] > distance[i][k] + distance[k][j]
            distance[i][j] ← distance[i][k] + distance[k][j]
         end if
```



Dijkstra algorithm

- Each node has 3 status: *unsearched*, *active*, *scanned*
- Label d(v) for every node v

```
Set all nodes v≠s unsearched and d(v)=+∞
Set s active and d(s)=0
While there is an active node
        Select an active u with minimum d(u)
        mark u scanned
        For every outgoing edge (u,v)
                d(v)← min{d(v), d(u)+w(u,v)}
                if v is unsearched then mark v active.
```

- Time: O(m+n log n) for every source s *(by Fibonacci heap)*
- APSP: $O(mn+n^2 \log n)$
  - When $m=O(n^2)$, running time is $O(n^3)$



Faster algorithms for $m=\Theta(n^2)$?

- All-Pairs Shortest Paths with integer edge weights in [-M,M].

Problem	Running time	Authors
Any graph	$O(mn+n^2 \log\log n)$	Pettie 2004
Undirected unweighted APSP	$O(n^{\omega})=O(n^{2.38})$	Seidel '95
Undirected APSP	$O(Mn^{2.38})$	Shoshan-Zwick '99
Directed APSP	$O(M^{0.68}n^{2.58})$	Zwick '98
(1+ε)-Approximate APSP	$O(n^{2.38} \log M/\varepsilon)$	Zwick '98

- No APSP algorithm in $O(n^{3-\delta})$ time for constant $\delta>0$ when $M=\Omega(n)$



Progress on real-weighted APSP

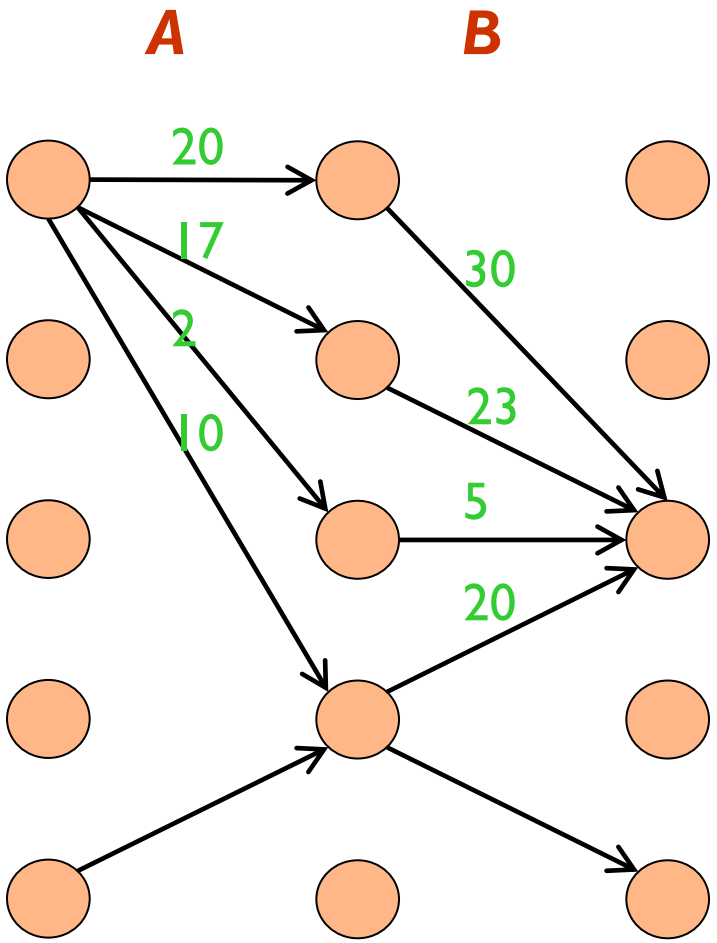
- What we want:
- Truly subcubic algorithm
  - For edge weights in [-M, M]
  - Runs in $O(n^{3-\delta} \cdot \text{polylog } M)$ time for constant $\delta > 0$
- Still OPEN!

Lots of efforts to it:

Author	Runtime	Year
Fredman	$n^3 \log\log^{1/3} n / \log^{1/3} n$	1976
Takaoka	$n^3 \log\log^{1/2} n / \log^{1/2} n$	1992
Dobosiewicz	$n^3 / \log^{1/2} n$	1992
Han	$n^3 \log\log^{5/7} n / \log^{5/7} n$	2004
Takaoka	$n^3 \log\log^2 n / \log n$	2004
Zwick	$n^3 \log\log^{1/2} n / \log n$	2004
Chan	$n^3 / \log n$	2005
Han	$n^3 \log\log^{5/4} n / \log^{5/4} n$	2006
Chan	$n^3 \log\log^3 n / \log^2 n$	2007
Han, Takaoka	$n^3 \log\log n / \log^2 n$	2012
Williams	$n^3 / \exp(\sqrt{\log n})$	2014



An interesting special case of the APSP problem



$$C = A * B$$

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

Min-Plus product
/Distance product



Algebraic Products

$$C = A\,B$$

$$c_{ij} = \sum_k a_{ik} b_{kj}$$

Current bound is $O(n^{2.3714})$

[Alman, Duan, V. Williams, Xu, Xu, Zhou 2025]

Distance Products

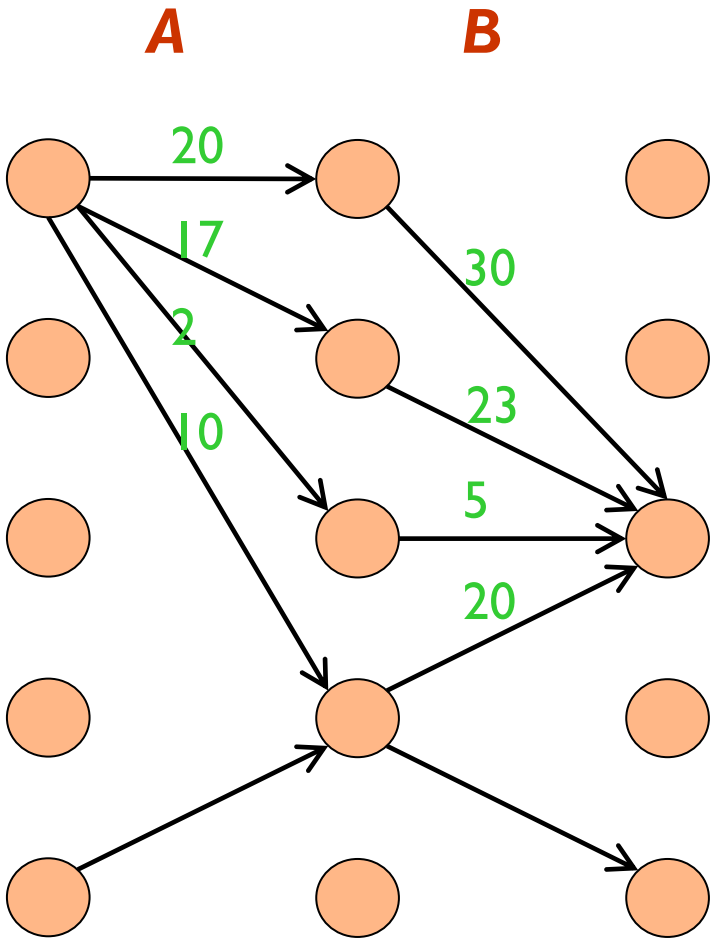
$$C = A * B$$

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

Currently no $O(n^{2.999})$ time algorithm!



An interesting special case of the APSP problem



$$C = A * B$$

$$c_{ij} = \min_k \{a_{ik} + b_{kj}\}$$

So distance product reduces to APSP:

- if we can solve APSP in T(n) time, we can solve distance product in T(n) time



APSP $\Longrightarrow$ Distance product

If W is an n by n matrix containing the edge weights of a graph.
Then W^n is the distance matrix

By induction, W^k gives the distances realized by paths that use at most k edges

$D \leftarrow W$
for $i \leftarrow 1$ to $\lceil \log_2 n \rceil$
do $D \leftarrow D*D$

Thus: APSP(n) ≤ MPP(n) log n

- The APSP and distance product are equivalent (for truly sub-cubic algorithms)



Easier problem?

- finding a minimum/maximum weight triangle
- or decision problem: is there a negative weight triangle?

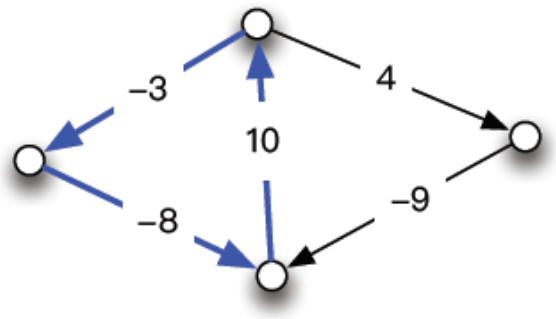
Input: Graph G with integer edge weights

Output:

Yes' if there exist nodes i,j,k in G such that

$$w(i,j) + w(j,k) + w(k,i) < 0$$

No' otherwise.





Negative triangle

- Trivial $O(n^3)$ algorithm
- in general, no $O(n^{3-\delta})$ time algorithm for constant $\delta>0$ known

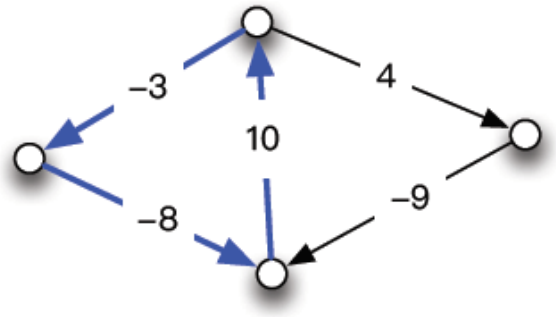
Input: Graph G with integer edge weights

Output:

Yes' if there exist nodes i,j,k in G such that

$$w(i,j) + w(j,k) + w(k,i) < 0$$

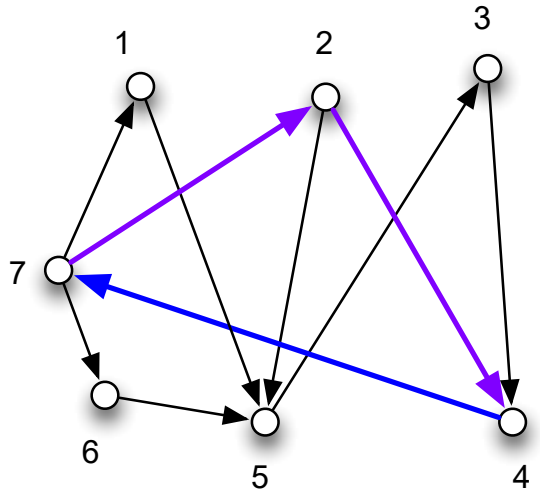
No' otherwise.





Negative triangle ⟹ Distance Product

- If we can compute the distance product of A*A, where A is the adjacency matrix of G
  - then for all pairs of (i,j), we just need to check whether (A*A)[i,j]+A[j,i]<0
  - so negative triangle reduces to distance product
  - seems a little bit easier?





Matrix multiplication verification

- $(+, \times)$-product
- Given $n \times n$ matrices A, B, C, verify whether $A \cdot B = C$

Freivalds' algorithm

1. Generate an $n \times 1$ random 0/1 vector r
2. Compute $P = A(B \cdot r) - C \cdot r$
3. Return "YES" if P is all zero, otherwise return "NO"



Matrix multiplication verification

- $(+, \times)$-product
- Given $n \times n$ matrices A, B, C, verify whether $A \cdot B = C$

Freivalds' algorithm

1. Generate an $n \times 1$ random 0/1 vector r
2. Compute $P = A(B \cdot r) - C \cdot r$
3. Return "YES" if P is all zero, otherwise return "NO"

- it's easy to see if $A \cdot B = C$, always return "yes", if $A \cdot B \neq C$, return "yes" with prob. $\leq 1/2$
- only matrix-vector multiplication, so running time: $O(n^2)$



OPEN Problems

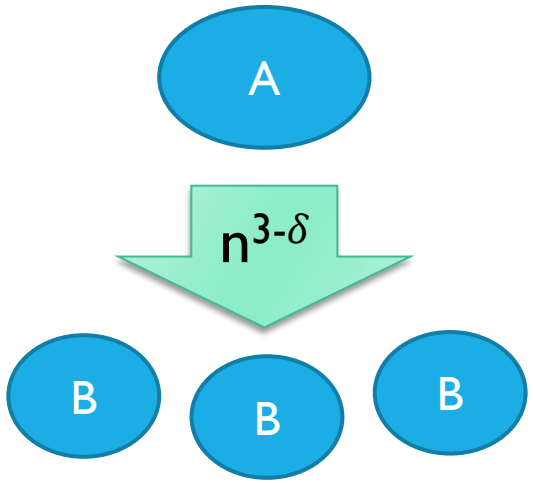
- Does *negative triangle problem* have a truly subcubic time algorithm?

- Does *distance product* have a truly subcubic **verification** algorithm?

truly subcubic: $O(n^{3-\delta} \cdot \text{polylog } M)$ for (integer) weights in [-M,M]



- *[Vassilevska, Williams 2010]* The following problem either all have truly subcubic time algorithms, or none of them do
  - all pair shortest path
  - distance product
  - negative triangle
  - distance product verification
  - minimum weight cycle
  - replacement path
  - …

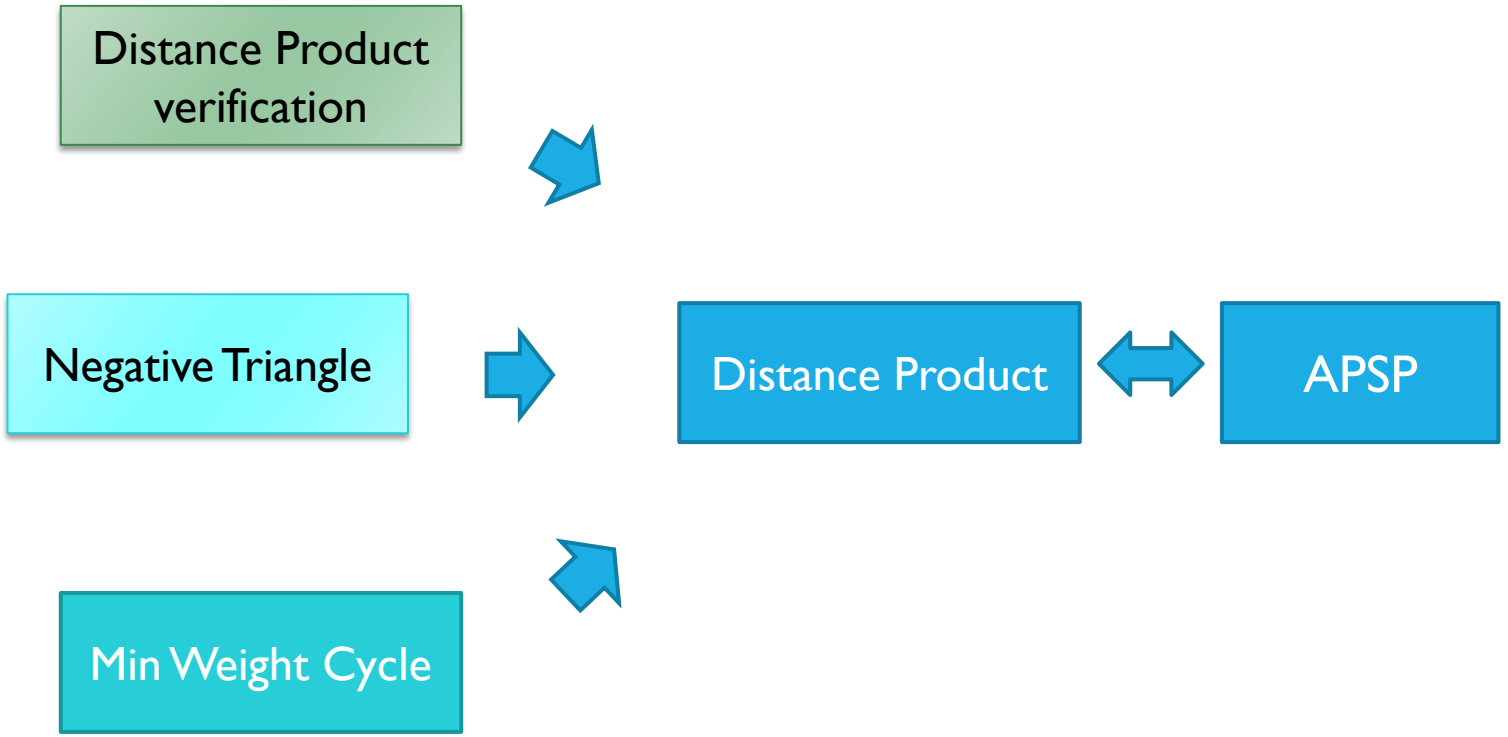


B instance sizes $n_1, \ldots, n_k$ so that $\Sigma_i n_i^{3-\varepsilon} < n^{3-\delta}$

- (by subcubic reductions)



Easy reductions





Distance Product ⟹ Negative Triangle

- Distance product comparison problem:

  given A,B,C, for every pair of i,j, check whether (A*B)[i,j]<C[i,j]

- First reduce distance product to distance product comparison:

Simultaneous binary search for C[i,j]

CLAIM: If $n \times n$ distance product comparison can be solved in $T(n)$ time ($\Omega(n^2)$),

then $n \times n$ distance product can be solved in $O(T(n) \cdot \log M)$ time



Distance Product ⟹ Negative Triangle

- First reduce distance product to distance product comparison: ✅

  given A,B,C, for every pair of i,j, check whether (A*B)[i,j]<C[i,j]

- then reduce distance product comparison to negative triangle
  - by a truly subcubic time algorithm for finding one negative triangle, design a truly subcubic time algorithm to compare all-pair distances?



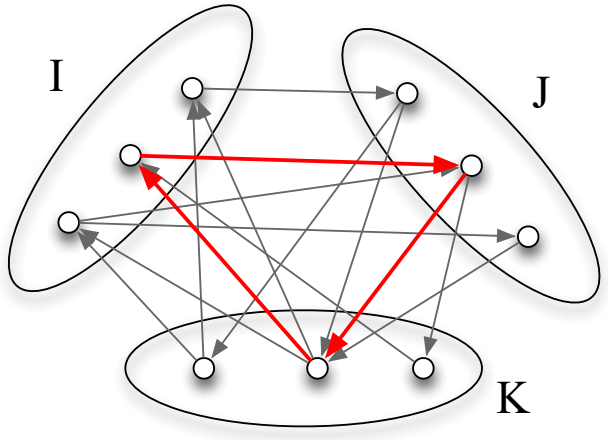


Distance product comparison ⟹ negative triangle

- W.l.o.g. suppose G is a tripartite graph $I \times J \times K$
- for $i \in I$, $j \in J$, $k \in K$, $w(k,i)=A[i,k]$, $w(j,k)=B[k,j]$, $w(i,j)=-C[i,j]$

- Suppose negative triangle has a $T(n)$-time algorithm

```
Repeat
    find a negative triangle (i,j,k)
        // then A[i,k]+B[k,j]-C[i,j]<0 and (A*B)[i,j]<C[i,j]
    delete (i,j)
Until (I×J=∅) or (no negative triangle exists)
```



For edge (i,j), if there is no negative triangle contains it, then $A[i,k]+B[k,j]-C[i,j] \geq 0$ for all k, so $(A*B)[i,j] \geq C[i,j]$

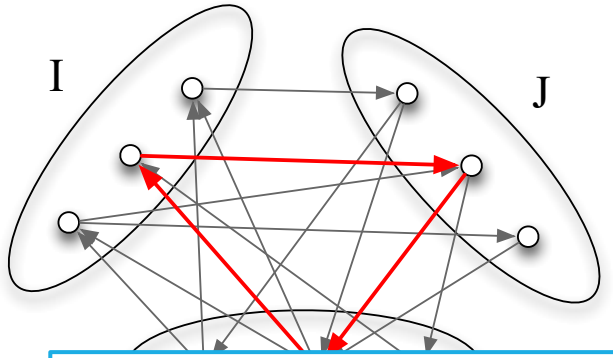


Distance product comparison ⟹ negative triangle

- W.l.o.g. suppose G is a tripartite graph $I \times J \times K$
- for $i \in I$, $j \in J$, $k \in K$, $w(k,i)=A[i,k]$, $w(j,k)=B[k,j]$, $w(i,j)=-C[i,j]$

- Suppose negative triangle has a $T(n)$-time algorithm

```
Repeat
    find a negative triangle (i,j,k)
        // then A[i,k]+B[k,j]-C[i,j]<0 and (A*B)[i,j]<C[i,j]
    delete (i,j)
Until (I×J=∅) or (no negative triangle exists)
```



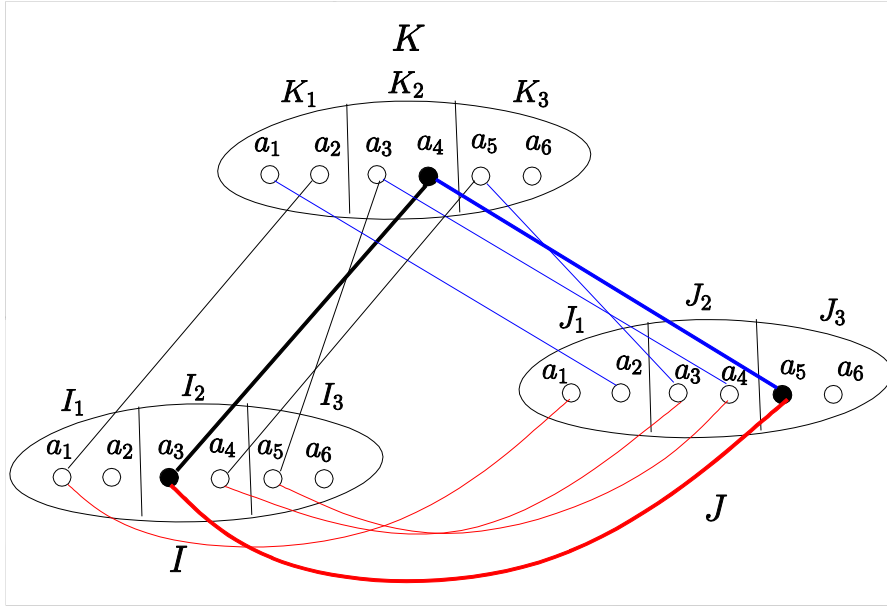
"all-pair negative triangle": for all pair (i,j), whether there is a negative triangle (i,j,k)

So we can get an $O(T(n) \cdot n^2)$-time distance product comparison algorithm



Distance product comparison ⟹ negative triangle

- What about split I, J, K into pieces of size s
  - number of pieces: O(n/s)
- consider all $(n/s)^3$ triples of I_x, J_y, K_z
  - finding a negative triangle: T(s) time



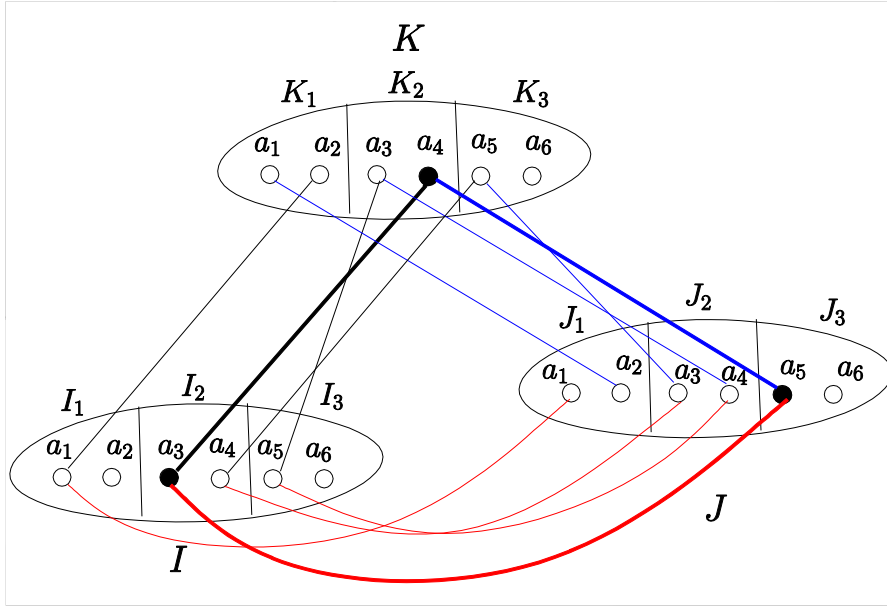


Distance product comparison ⟹ negative triangle

- What about split I, J, K into pieces of size s
  - number of pieces: O(n/s)
- consider all $(n/s)^3$ triples of I_x, J_y, K_z
  - finding a negative triangle: T(s) time

For every triple I_x, J_y, K_z
  while I_x, J_y, K_z has a negative triangle
    find a negative triangle (i,j,k)
      *// then (A*B)[i,j]<C[i,j]*
    delete (i,j)

- Total time?

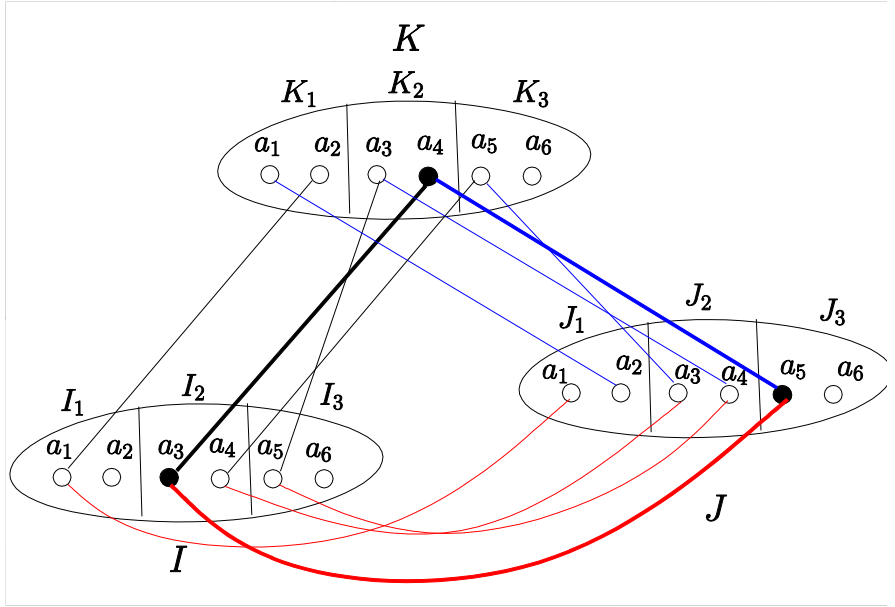




Distance product comparison ⟹ negative triangle

- What about split I, J, K into pieces of size s
  - number of pieces: O(n/s)
- consider all $(n/s)^3$ triples of I_x, J_y, K_z
  - finding a negative triangle: T(s) time

For every triple I_x, J_y, K_z
  while I_x, J_y, K_z has a negative triangle
    find a negative triangle (i,j,k)
      *// then (A*B)[i,j]<C[i,j]*
    delete (i,j)



TIME: $O((n/s)^3+n^2)\cdot T(s))$

- every call of negative triangle: T(s)
- total number of successful calls (each time delete an (i,j)-pair): $O(n^2)$
- total number of unsuccessful calls: $O((n/s)^3)$



Distance product comparison ⟹ negative triangle

- What about split I, J, K into pieces of size s
  - number of pieces: O(n/s)
- consider all $(n/s)^3$ triples of I_x, J_y, K_z
  - finding a negative triangle: T(s) time

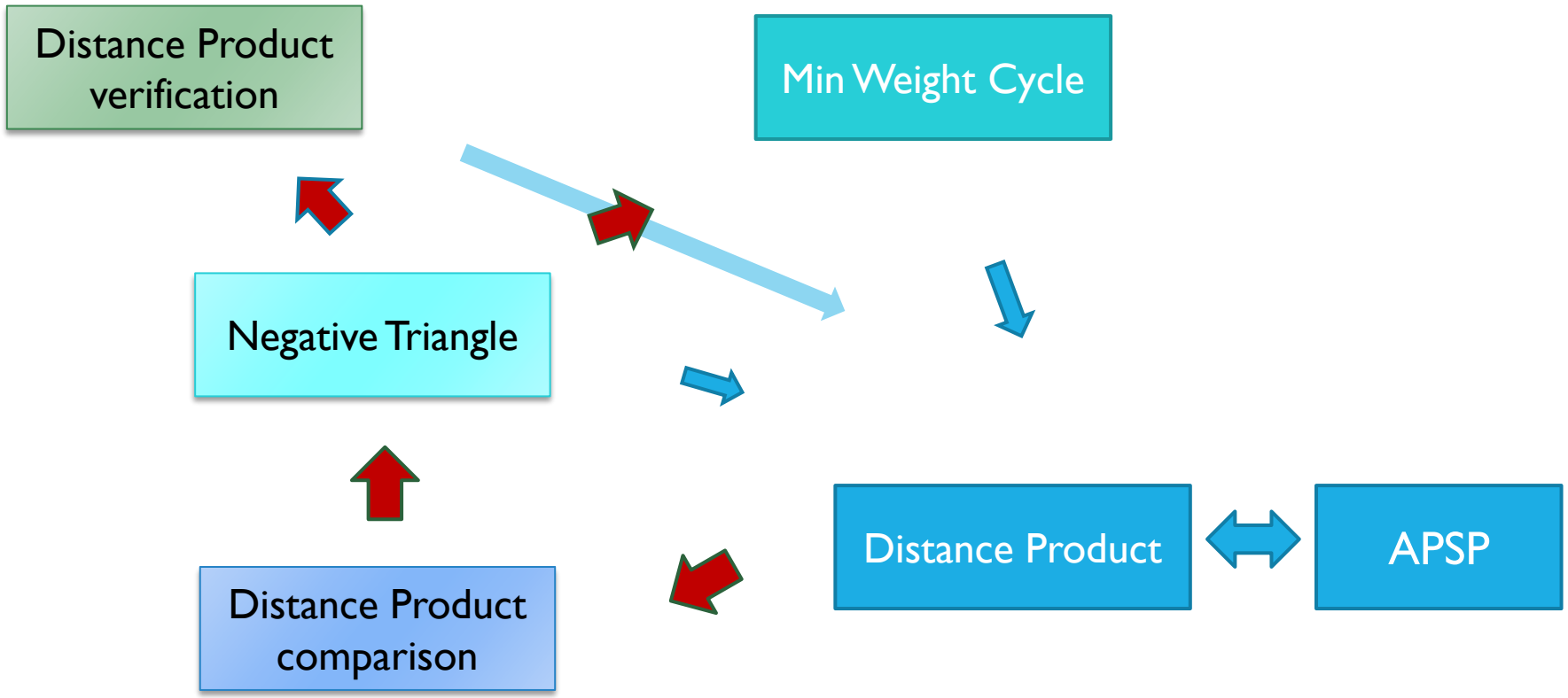
```
For every triple Ix, Jy, Kz
    while Ix, Jy, Kz has a negative triangle
        find a negative triangle (i,j,k)
            // then (A*B)[i,j]<C[i,j]
        delete (i,j)
```

TIME: $O((n/s)^3+n^2)\cdot T(s))$

If $T(n)=n^{3-\delta}$, let $s=n^{1/3}$, we will have an $O(n^{3-\delta/3})$ algorithm



Reductions


Distance Product verification
Min Weight Cycle
Negative Triangle
Distance Product comparison
Distance Product
APSP

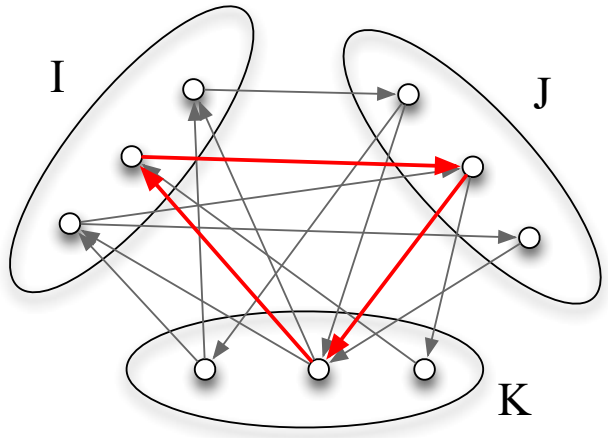
Negative triangle ⟹ Distance Product verification

- W.l.o.g. suppose G is a tripartite graph $I \times J \times K$
- for $i \in I$, $j \in J$, $k \in K$, $w(k,i)=A[i,k]$, $w(j,k)=B[k,j]$, $w(i,j)=-C[i,j]$

Define the identity matrix $\mathbb{I}$:

$$\mathbb{I}=\begin{bmatrix} 0 & \infty & \cdots & \infty \\ \infty & 0 & & \infty \\ \vdots & & \ddots & \vdots \\ \infty & \infty & \cdots & 0 \end{bmatrix},$$

then $\mathbb{I}*C=C$ for the distance product $*$



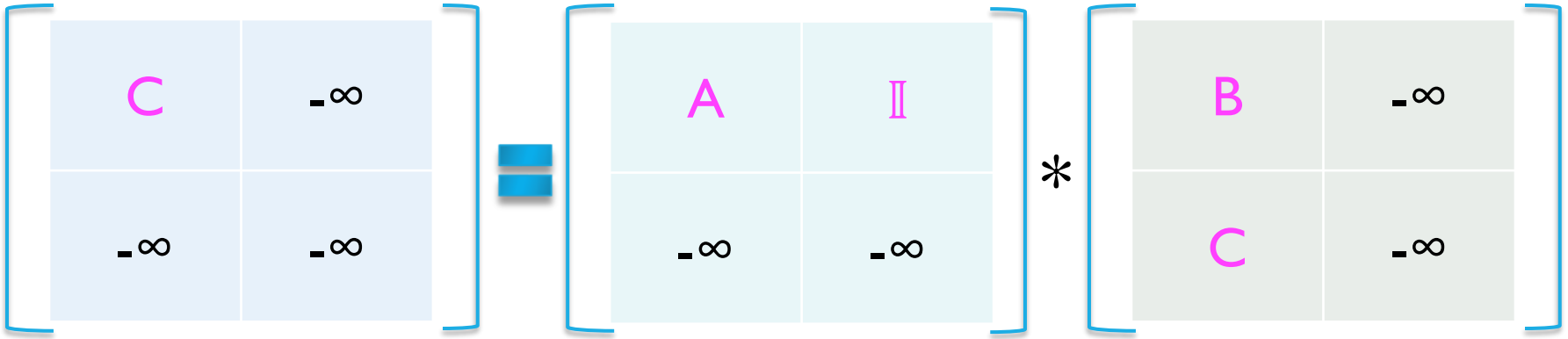


Negative triangle ⟹ Distance Product verification

- for i∈I, j∈J, k∈K, w(k,i)=A[i,k], w(j,k)=B[k,j], w(i,j)=-C[i,j]

Define the identity matrix $\mathbb{I}=\begin{bmatrix} 0 & \infty & \cdots & \infty \\ \infty & 0 & & \infty \\ \vdots & & \ddots & \vdots \\ \infty & \infty & \cdots & 0 \end{bmatrix}$,

- If we have an n×n distance product verification algorithm in time T(n), then we can check existence of a negative triangle by verifying C'=A'*B':





Negative triangle ⟹ Distance Product verification

- for i∈I, j∈J, k∈K, w(k,i)=A[i,k], w(j,k)=B[k,j], w(i,j)=-C[i,j]

Define the identity matri

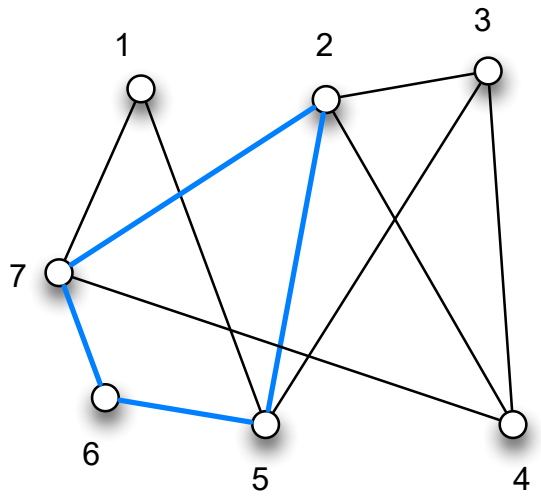
A negative triangle (i, j, k): A[i,k]+B[k,j]-C[i,j]<0
⟺ $\min_k\{A[i,k]+B[k,j]\}<C[i,j]$
⟺ ∃i,j, min{(A*B)[i,j], C[i,j]}≠C[i,j]
⟺ ∃i,j, A'*B'[i,j]≠C'[i,j]

- If we have an n×n distance product
eck existence of a negative triangle by verifying C'=A

$$\begin{bmatrix} C & -\infty \\ -\infty & -\infty \end{bmatrix} = \begin{bmatrix} A & \mathbb{I} \\ -\infty & -\infty \end{bmatrix} * \begin{bmatrix} B & -\infty \\ C & -\infty \end{bmatrix}$$



Negative triangle ⟹ Min Weight Cycle



Min-weight cycle in nonnegative weight graphs reduced to APSP

Min-weight simple cycle with negative weights?

NP-complete since Hamiltonian cycle reduced to it



Negative triangle ⟹ Min Weight Cycle

- If the edge weights in G are in [-M,M]
- add 8M to every weight, then edge weights are in [7M,9M]
  - then 4-cycle has weight ≥28M, larger than any 3-cycle ≤27M

- Thus, by the algorithm for minimum weight cycle, we can find the minimum weight triangle



APSP-hardness

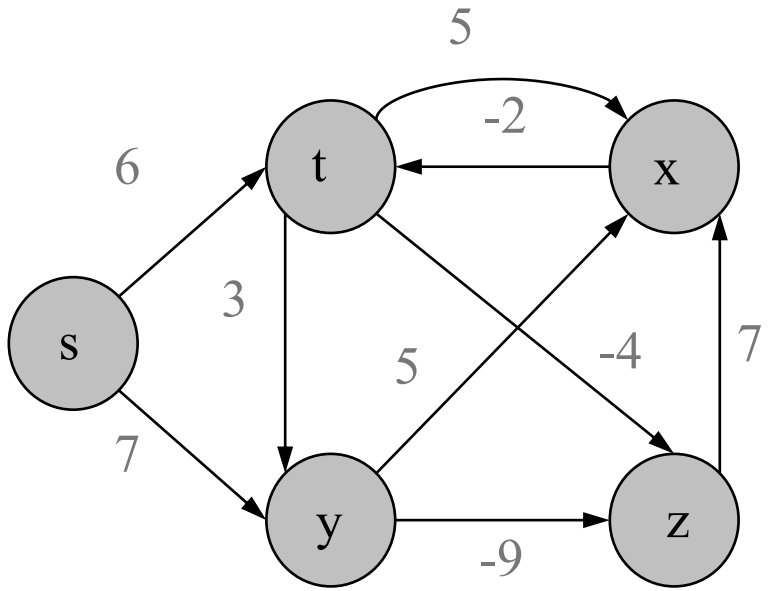
- Subcubic equivalent to APSP:
  - negative weight triangle
  - minimum weight triangle
  - minimum weight cycle in nonnegative weight graphs
- NP-complete:
  - minimum weight simple simple cycle in graphs with negative weights

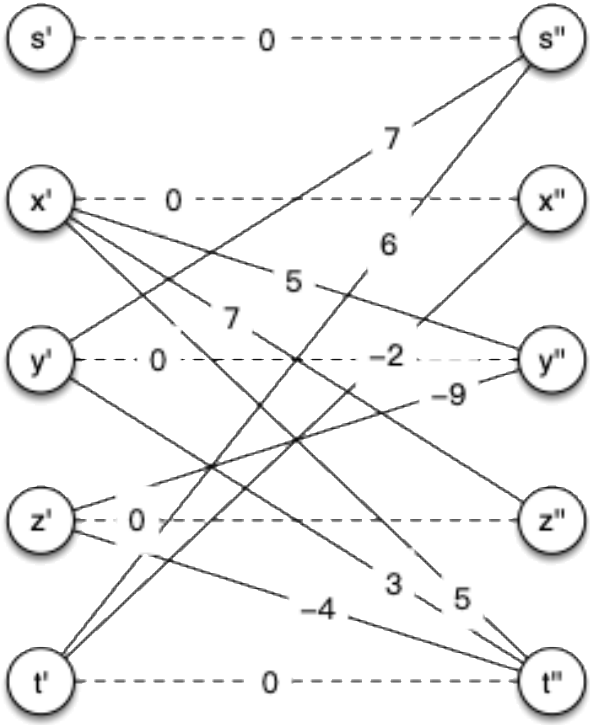
- What about negative weight cycle in graphs with negative weights?



Negative weight cycle to matching

- duplicate all vertex v to v' and v"
- an edge (u,v) → (u",v')
- (v',v") has weight 0

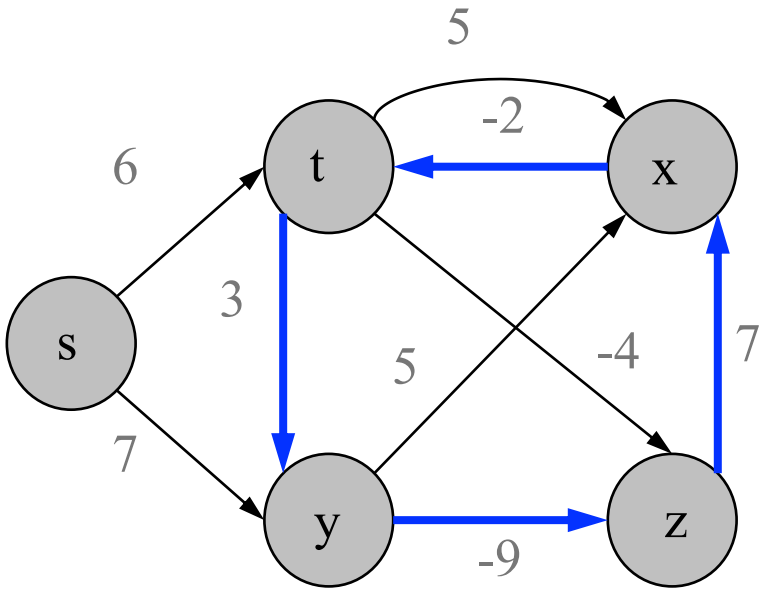


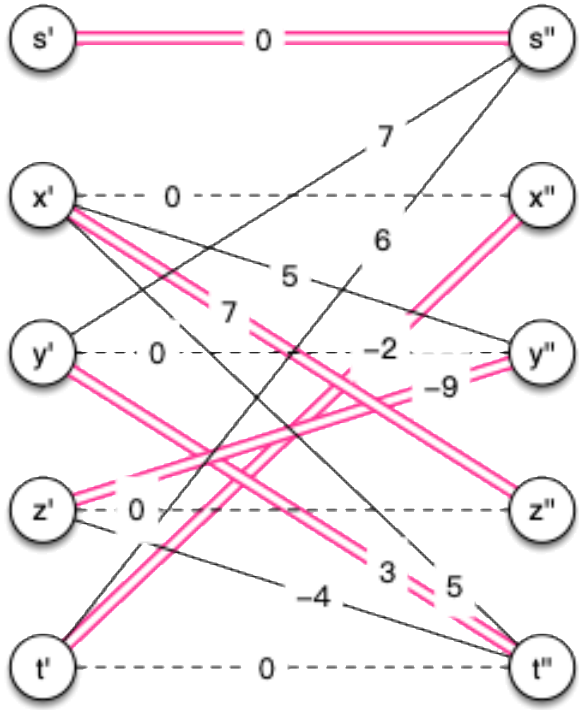




Negative weight cycle to matching

- a minimum weight perfect matching in the new graph corresponds to an negative weight cycle and some 0-weight edges







APSP-hardness

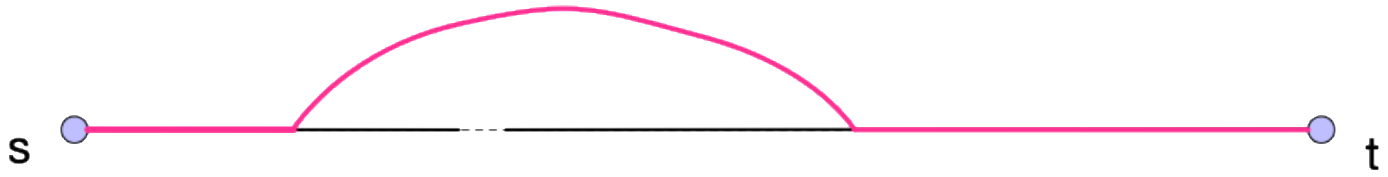
- Subcubic equivalent to APSP:
  - negative weight triangle
  - minimum weight triangle
  - minimum weight cycle in nonnegative weight graphs
- NP-complete:
  - minimum weight simple simple cycle in graphs with negative weights
- Subcubic time:
  - negative weight cycle in graphs with negative weights
    - $O(mn^{1/2} \log(nW))$ algorithm *[Gabow & Tarjan 1987]*
    - near-linear time min-cost flow algorithm *[Chen et al. 2022]*
    - strongly subcubic time: $\tilde{O}(n^{2.5})$ *[Li, Li, Rao, Zhang 2026]*



Replacement path & Second shortest path

- Replacement path
  - Given s,t and shortest path P from s to t
  - Find shortest path from s to t in G-{e}, for all edge e∈P

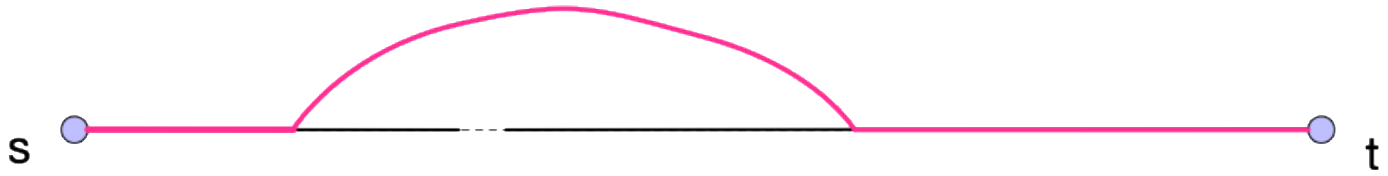
  - Second shortest path: shortest among all replacement paths





Replacement path ⟹ APSP

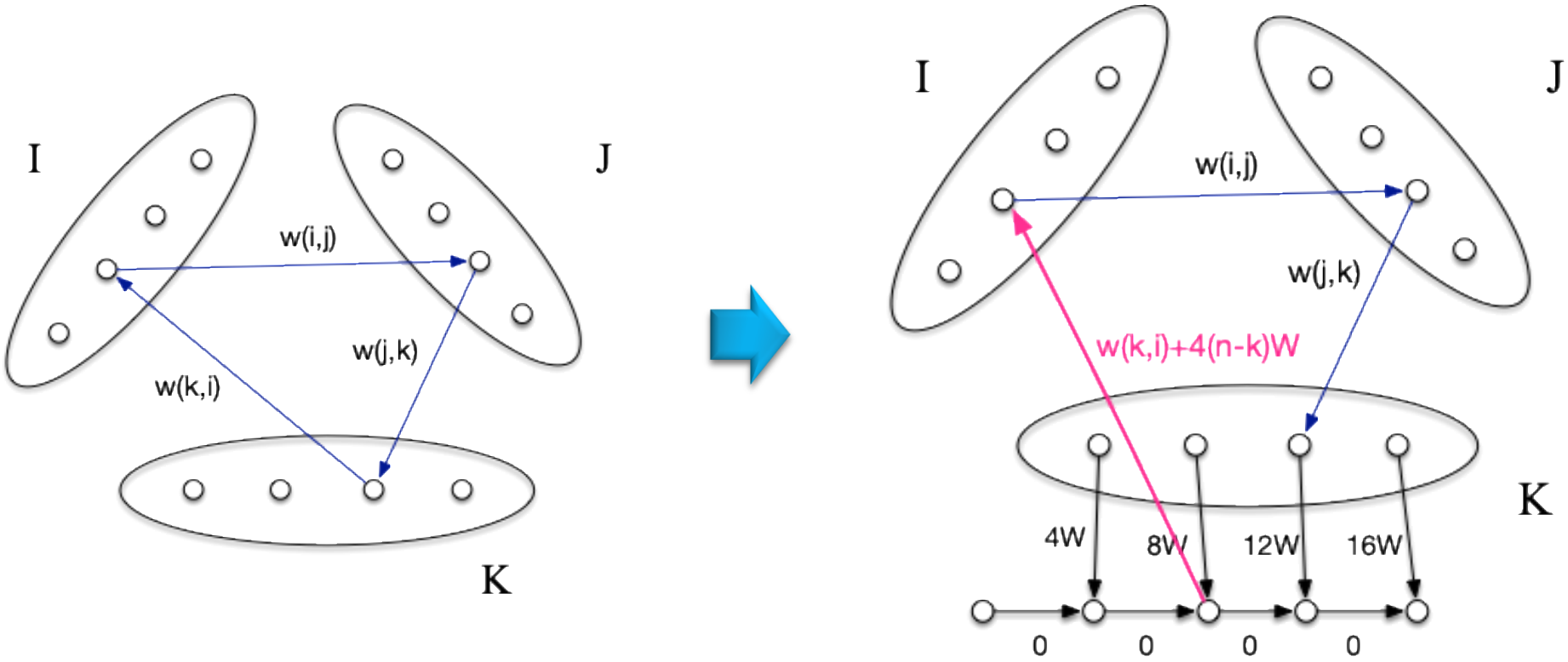
- Replacement path
  - Delete all edges in P, compute all-pair shortest paths between vertices in P
  - Use segment tree structures…
  - Second shortest path: shortest among all replacement paths
  - So second shortest path ⟹ replacement paths





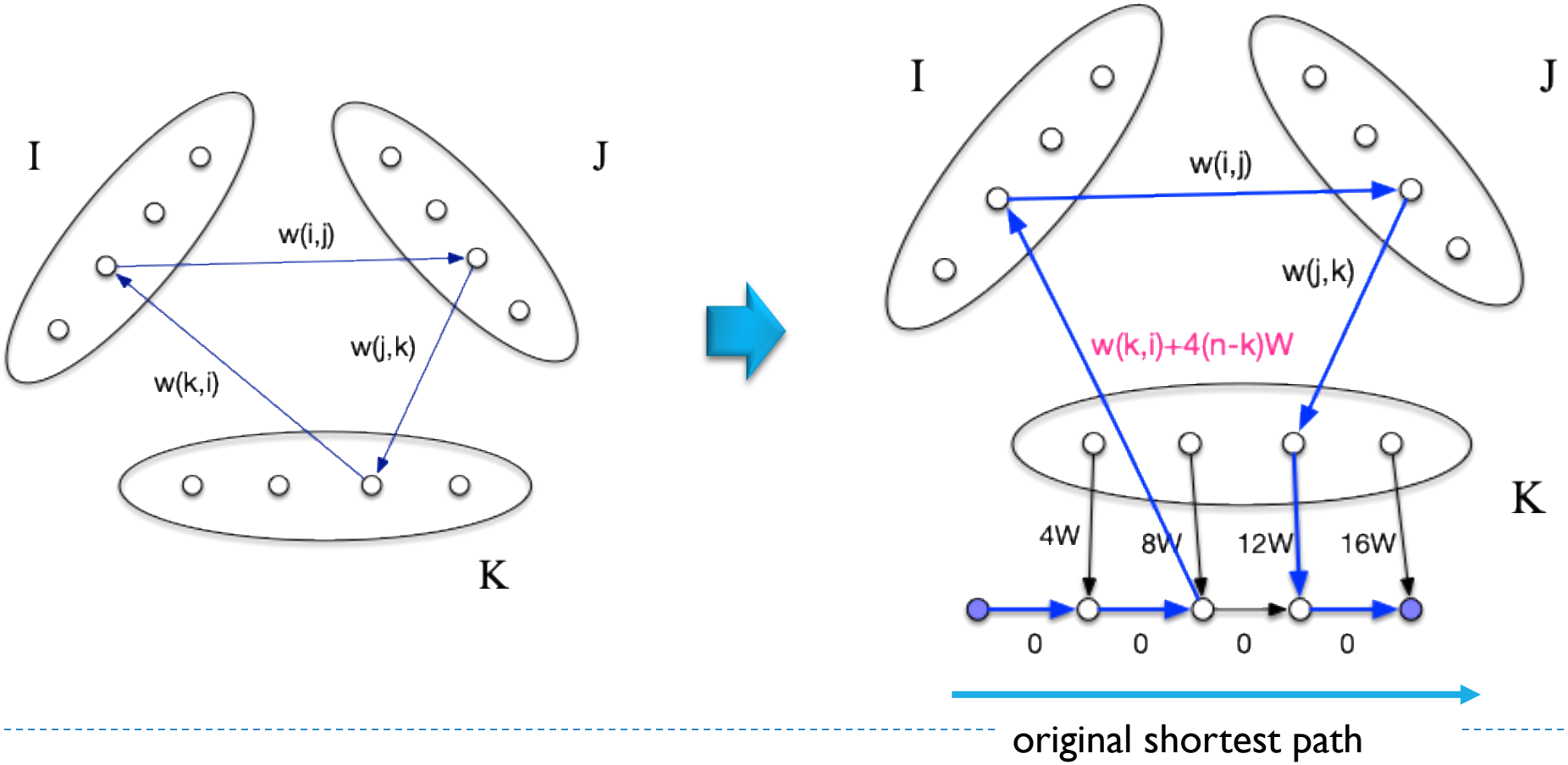
Min-weight triangle ⟹ Second shortest path

- Find a min-weight triangle in $I \times J \times K$:



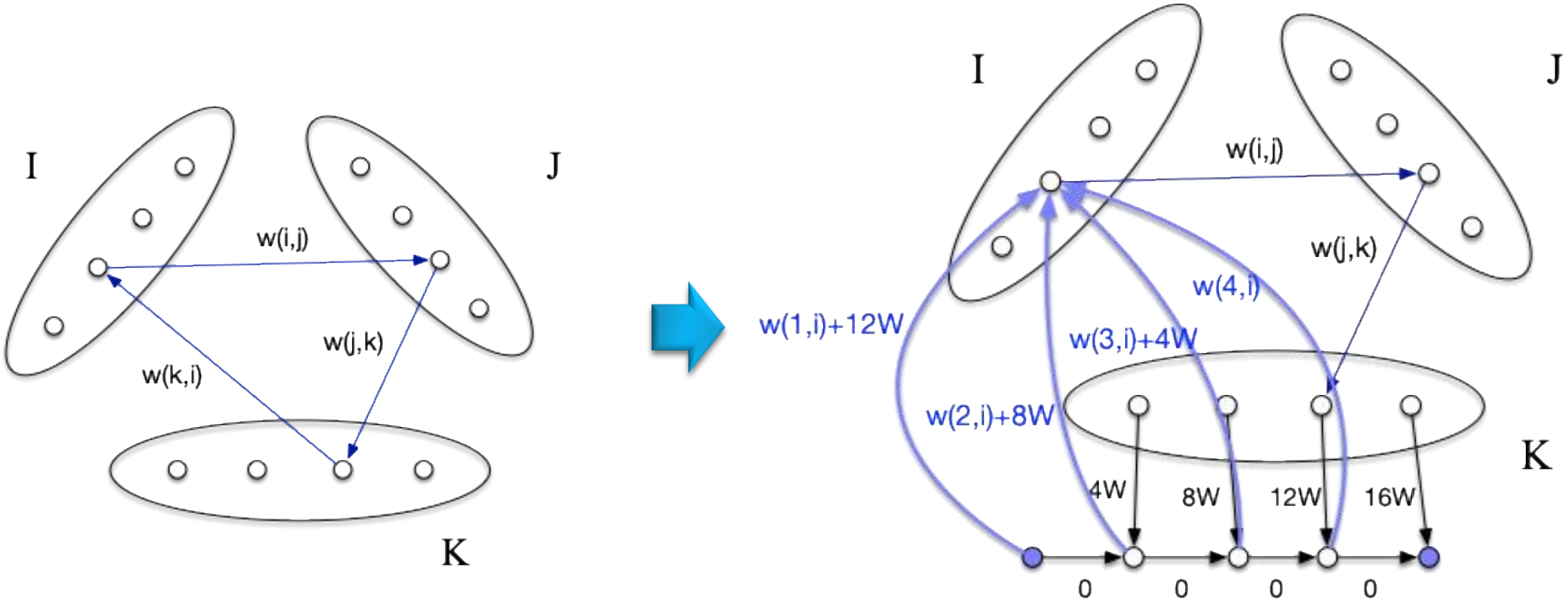
Min-weight triangle ⟹ Second shortest path

- Find a min-weight triangle in $I \times J \times K$:



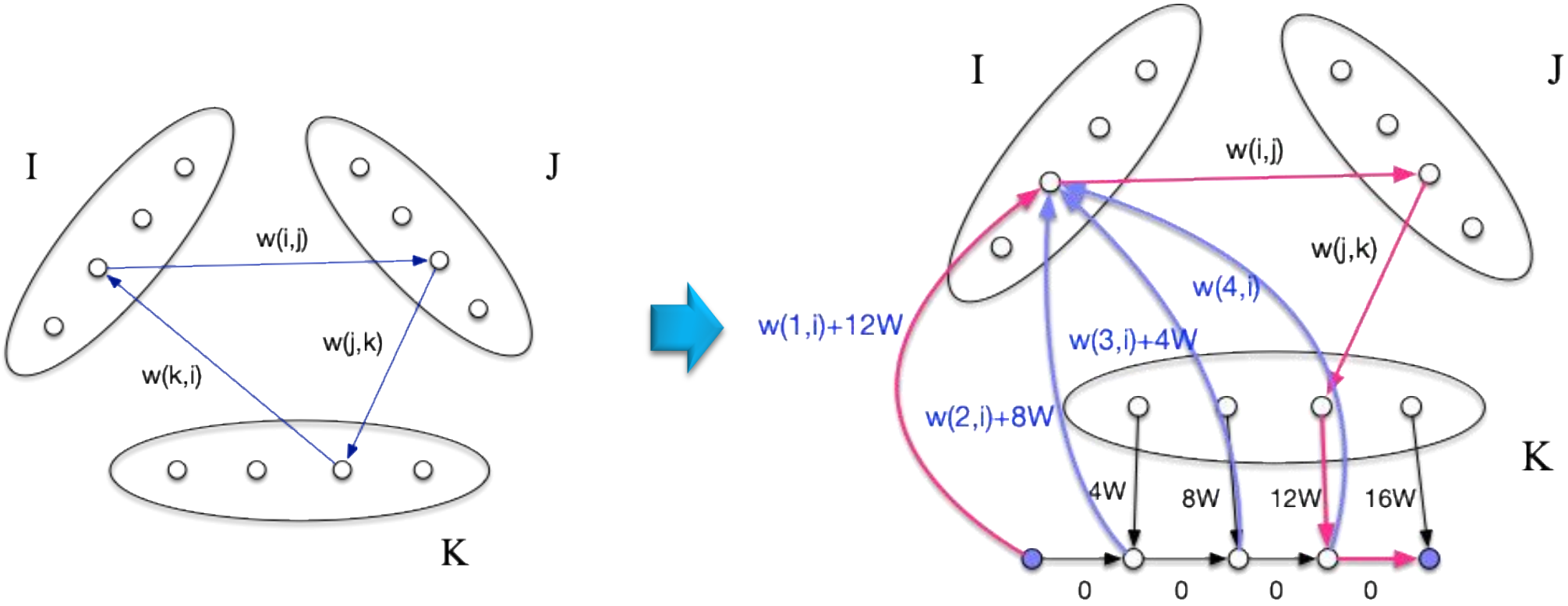
Min-weight triangle ⟹ Second shortest path

- Find a min-weight triangle in $I \times J \times K$:



Min-weight triangle ⟹ Second shortest path

- Find a min-weight triangle in $I \times J \times K$:



so second shortest path must correspond to triangle in original graph



Next-to-shortest path

- Second shortest path problem find shortest one different than a shortest path
  - but it can have the same length as shortest path

- Next-to-shortest path: shortest simple path among all not shortest s-t paths
  - or simply "not-shortest-path"
  - believed to be NP-complete
  - $\tilde{O}(n^4m^3)$-time algorithm *[Chen, Wein, Zhang 2025]*



APSP-hardness

- *[Vassilevska, Williams 2010]* The following problem either all have truly subcubic time algorithms, or none of them do
  - all pair shortest path
  - distance product
  - negative triangle
  - distance product verification
  - minimum weight cycle
  - replacement path
  - second shortest path
  - …



Trivial algorithms?

3SUM: Given a set S of integers, $|S|=n$ are there $a,b,c \in S$ so that $a+b+c=0$?

Orthogonal vector (OV): Given a set S of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $u,v \in S$ with $u \cdot v=0$?

All pair shortest path (APSP): Given a weighted graph, find the distance between every pair of vertices

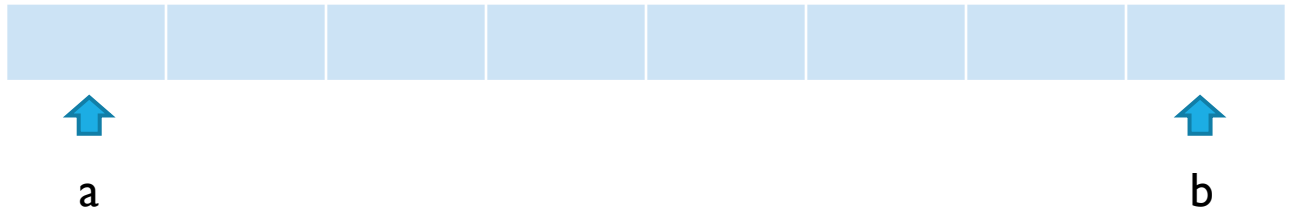


Trivial algorithm?

3SUM:

Given a set S of integers, |S|=n
are there a,b,c∈S so that a+b+c=0?

- sort all elements in S
- for all c∈S, find a+b=-c
- TIME: $O(n^2)$



if a+b>-c, then b goes left
if a+b<-c, then a goes right



3-SUM

Given a set S of integers, $|S|=n$
are there $a,b,c \in S$ so that $a+b+c=0$?

- Easy $O(n^2)$ time algorithm
- *[Baran, Demaine, Pătraşcu 2005]* $\sim n^2/\log^2 n$ time algorithm for integers
- *[Grønlund, Pettie 2014]* $\sim n^2/\log n$ time algorithm for real numbers
- Folklore: we consider 3SUM over integers in $\{-n^3,\ldots,n^3\}$

3SUM Conjecture: 3SUM on n integers in $\{-n^3,\ldots,n^3\}$ requires $n^{2-o(1)}$ time.



Orthogonal vector (OV)

Given a set S of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $u,v \in S$ with $u \cdot v = 0$?

- Trivial $O(n^2 \log n)$ time algorithm
- *[Abboud, Williams, Yu 2015]*: $n^{2-\Theta\left(\frac{1}{\log(d/\log n)}\right)}$

OV Conjecture: OV on n vectors requires $n^{2-o(1)}$ time.



Exponential time hypothesis (ETH)

- Conjecture: 3-SAT cannot be solved in subexponential time

- ETH: 3-SAT requires $2^{\delta n}$ time for some constant $\delta>0$
  - n: #variables
  - m: #clauses

Not ETH Zürich

Eidgenössische Technische Hochschule Zürich





Simple $2^{(1-\varepsilon)n}$ time algorithm for 3-SAT

Schöning's algorithm

1. Pick random initial assignment x
2. While there is at least one unsatisfied clause and have done this ≤n times:
   a) Pick an arbitrary unsatisfied clause c
   b) Flip the bit of a random x_p in the clause



Simple $2^{(1-\varepsilon)n}$ time algorithm for 3-SAT

Schöning's algorithm

1. Pick random initial assignment x
2. While there is at least one unsatisfied clause and have done this ≤n times:
   a) Pick an arbitrary unsatisfied clause c
   b) Flip the bit of a random x_p in the clause

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee \neg x_4) \ldots$$

x = 1, 0, 1,0, …



Simple $2^{(1-\varepsilon)n}$ time algorithm for 3-SAT

Schöning's algorithm

1. Pick random initial assignment x
2. While there is at least one unsatisfied clause and have done this ≤n times:
   a) Pick an arbitrary unsatisfied clause c
   b) Flip the bit of a random x_p in the clause

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee \neg x_4) \ldots$$

x = 1, 0, 1, 0, …
x = 1, 0, 1, 1, …



Simple $2^{(1-\varepsilon)n}$ time algorithm for 3-SAT

Schöning's algorithm

1. Pick random initial assignment x
2. While there is at least one unsatisfied clause and have done this ≤n times:
   a) Pick an arbitrary unsatisfied clause c
   b) Flip the bit of a random x_p in the clause

Analysis:

- Given a fixed satisfying assignment α, the prob. that x differs from α by k bits is $\binom{n}{k}2^{-n}$
- When x differs from α by k bits, the prob. that every step we make a correct modification is 1/3
  - So after k steps, the prob. of getting correct α is $\left(\frac{1}{3}\right)^k$
  - Sum over k: $\sum_{k=0}^{n}\binom{n}{k}2^{-n}\left(\frac{1}{3}\right)^k = \left(\frac{1}{2}\left(1+\frac{1}{3}\right)\right)^n$
- Repeat for $O(1.5^n)$ times



Exponential time hypothesis (ETH)

- Conjecture: 3-SAT cannot be solved in subexponential time

- ETH: 3-SAT requires $2^{\delta n}$ time for some constant δ>0
  - (no $2^{O(n^{0.99})}$ time algorithm)
  - n: #variables
  - m: #clauses

- SETH: for every constant ε>0, there is a k such that k-SAT cannot be solved in $2^{(1-\varepsilon)n} \cdot \text{poly}(m)$ time
  - (no 1.99^n time algorithm for all k-SAT)
  - but k-SAT has $O((2-2/k)^n)$ time algorithm *[Schöning 1999]*



SETH $\Longrightarrow$ Orthogonal Vector (OV)

- Given a k-CNF: (n: #variables, m:#clauses)

  $(x_1 \vee \sim x_2 \vee \sim x_3) \wedge (\sim x_1 \vee \sim x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee \sim x_4) \ldots$

- By the algorithm of OV with $2^{n/2}$ vectors?
  - We only consider the case m=O(n)

> OV: Given a set S of n vectors in $\{0,1\}^d$, for d=O(log n), are there $u,v \in S$ with $u \cdot v = 0$?

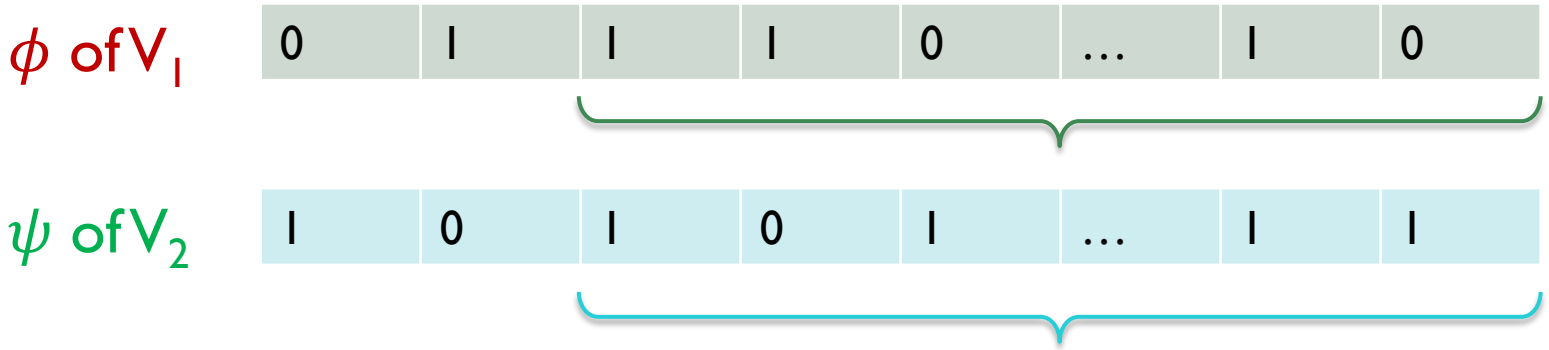


SETH $\Longrightarrow$ Orthogonal Vector (OV)

- Given a k-CNF: (n: #variables, m:#clauses)

  $(x_1 \vee \sim x_2 \vee \sim x_3) \wedge (\sim x_1 \vee \sim x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee \sim x_4) \ldots$

- We partition the variables into V_1, V_2 on n/2 variables each
- For all assignment ϕ, ψ of V_1, V_2, resp., construct (m+2)-vectors:



0 if ϕ/ψ satisfy the clauses, 1 otherwise



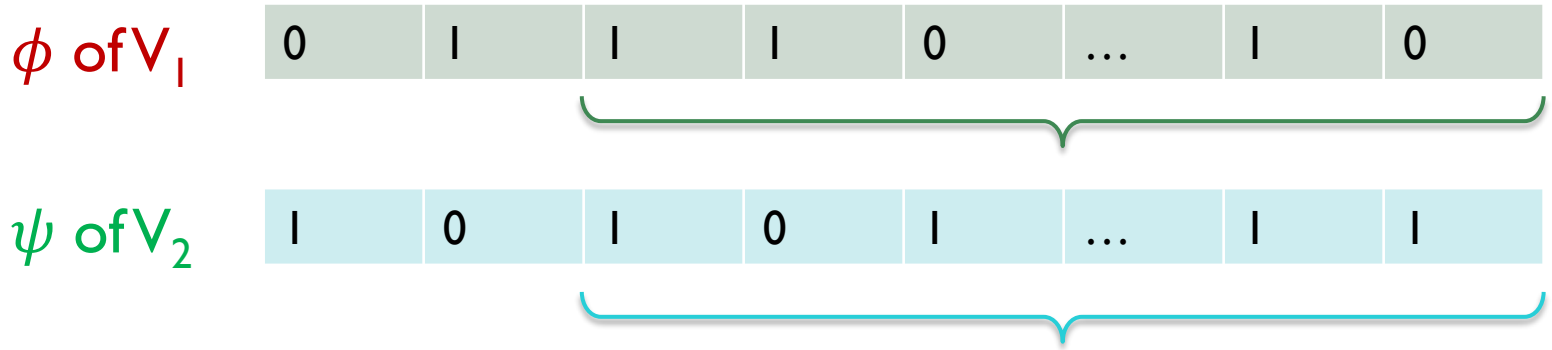
SETH ⟹ Orthogonal

- Given a k-CNF: (n: #variables, r

$(x_1 \vee \sim x_2 \vee \sim x_3) \wedge (\sim x_1 \vee \sim x_3 \vee$

A clause $(x_i \vee \sim x_j \vee \sim x_k)$ for $x_i, x_j \in V_1$ and $x_k \in V_2$ is satisfied iff some literal is true
Thus, for an satisfying assignment of k-CNF, its partition on V_1 and V_2, ϕ and ψ, must satisfy all clauses
So the corresponding vectors of ϕ and ψ are orthogonal

- We partition the variables into V_1, V_2 on n/2 variables each.
- For all assignment ϕ, ψ of V_1, V_2, resp., construct (m+2)-vectors:



0 if ϕ/ψ satisfy the clauses, 1 otherwise



Example

- Given a k-CNF: (n: #variables, m:#clauses)

$$(x_1 \vee \sim x_2 \vee \sim x_3) \wedge (\sim x_1 \vee \sim x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee \sim x_4) \ldots$$

  - $V_1=\{x_1,x_2,\ldots\}, V_2=\{x_3,x_4,\ldots\}$
  - the (m+2)-vectors:

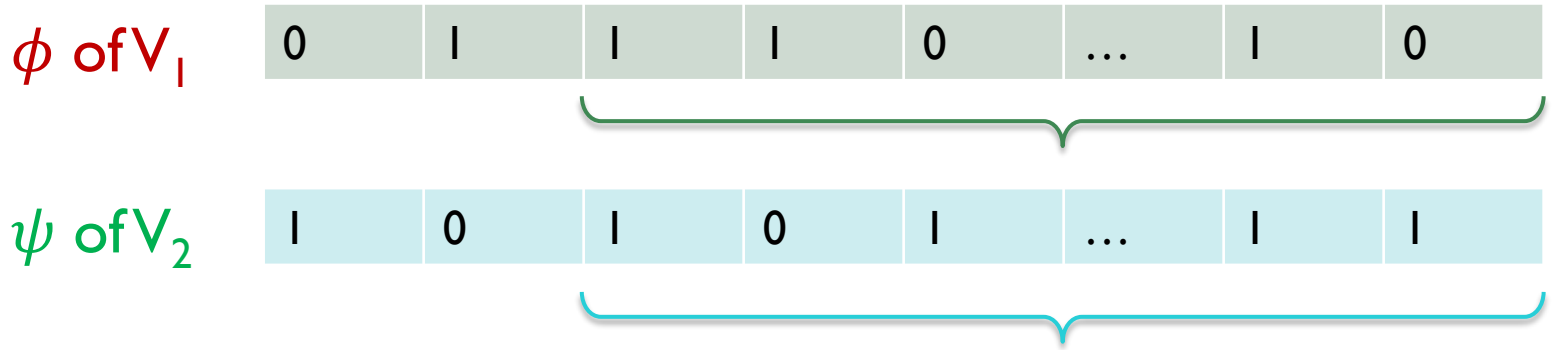
ϕ of V_1	Vector
00…	(0, 1, 0, 0, 1,…)
01…	(0, 1, 1, 0, 0,…)
10…	(0, 1, 0, 1, 1,…)
11…	(0, 1, 0, 1, 0,…)

ψ of V_2	Vector
00…	(1, 0, 0, 0, 0,…)
01…	(1, 0, 0, 0, 1,…)
10…	(1, 0, 1, 1, 0,…)
11…	(1, 0, 1, 0, 0,…)



OV Conjecture: OV on n vectors requires $n^{2-o(1)}$ time.

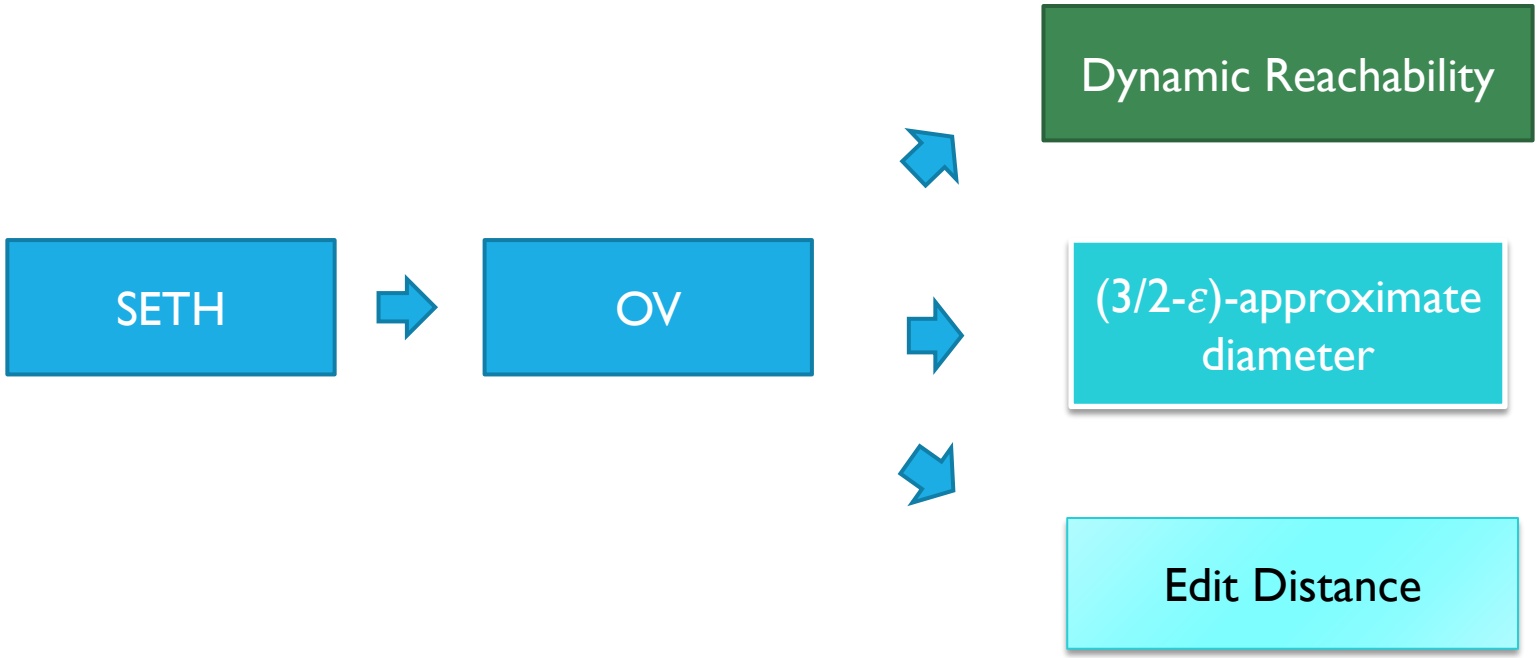
- So we have $N=2\cdot 2^{n/2}$ vectors with dimension $O(m)=O(n)$
- So if OV has an $N^{2-\delta}$ algorithm, we will have an $O(2^{n(1-\delta/2)})$ k-SAT algorithm



0 if ϕ/ψ satisfy the clauses, 1 otherwise



Reductions


SETH
OV
Dynamic Reachability
(3/2-ε)-approximate diameter
Edit Distance



Edit Distance

Edit Distance

- Two strings:

  AGGCTATCACCTGACCTCCAGGCCGATGCCC

  TAGCTATCACGACCGCGGTCGATTTGCCCGAC

- How similar are they? What do they have in common?



Edit Distance

- One edit operation on a string S:
  - **insert** a single symbol
  - **delete** a single symbol
  - **substitute** a symbol x in S with another symbol y

- **Edit Distance**: counting the minimum number of operations required to transform one string into the other

Basic algorithm?

AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC

-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTTGCCCGAC



Computing edit distance

- Dynamic programming takes $O(n^2)$ time
  - compare X[1…M] and Y[1…N]

```
For each  i = 1…M
      For each  j = 1…N
                          { D(i-1,j) + 1
          D(i,j)= min     { D(i,j-1) + 1
                          { D(i-1,j-1) +   1; { if X(i) ≠ Y(j)
                                          0; { if X(i) = Y(j)
```



Computing edit distance

- Dynamic programming takes $O(n^2)$ time
- Human genome: 3×10^9 base pairs
- A quadratic time algorithm is not fast!







Computing edit distance

- Dynamic programming takes $O(n^2)$ time
- Can be improved to $O(n^2/\log n)$ time [Masek, Paterson 1980]
- Better algorithms for special cases: [U83,LV85,M86, GG88,GP89,UW90,CL90,CH98,LMS98,U85,CL92,N99,CPSV00,MS00,CM02,BCF08,AK08,AKO10...]
- Approximation algorithm: constant approximation in near-linear time [Andoni & Nosatzki'18]
- …



Hardness

- Theorem: No $n^{2-\delta}$ time algorithm for edit distance unless SETH fails
  [Backurs,Indyk 2015]

- We've seen the reduction of SETH to orthogonal vector
- We'll show the reduction of OV to edit distance



OV $\Longrightarrow$ Edit Distance

- OV: Given sets A, B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a\in A$, $b\in B$ with $a\cdot b=0$?
  - Given $a=(a^1,\ldots,a^d)\in A$, $b=(b^1,\ldots,b^d)\in B$, we want $\sum_i a^i\cdot b^i=0$



OV $\Longrightarrow$ Edit Distance

- OV: Given sets A, B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a \in A$, $b \in B$ with $a \cdot b = 0$?
- Idea:
  - transform every vector into a short string
    - $a_i \in A \rightarrow \alpha_i$ $\quad |\alpha_i| \leq d^{O(1)}$
    - $b_j \in B \rightarrow \beta_j$ $\quad |\beta_j| \leq d^{O(1)}$
    - If a_i and b_j are orthogonal, then $edit(\alpha_i, \beta_j) = S$
    - If a_i and b_j are NOT orthogonal, then $edit(\alpha_i, \beta_j) = L$, $\quad S < L$
    - Crucial that L does not depend on vectors



OV $\Longrightarrow$ Edit Distance

- OV: Given sets A, B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a \in A$, $b \in B$ with $a \cdot b = 0$?
- Idea:
  - transform every vector into a short string
  - then put all strings of vectors in A and B together

• t – vector gadget for vector consisting of 1s

$d^{O(1)}$

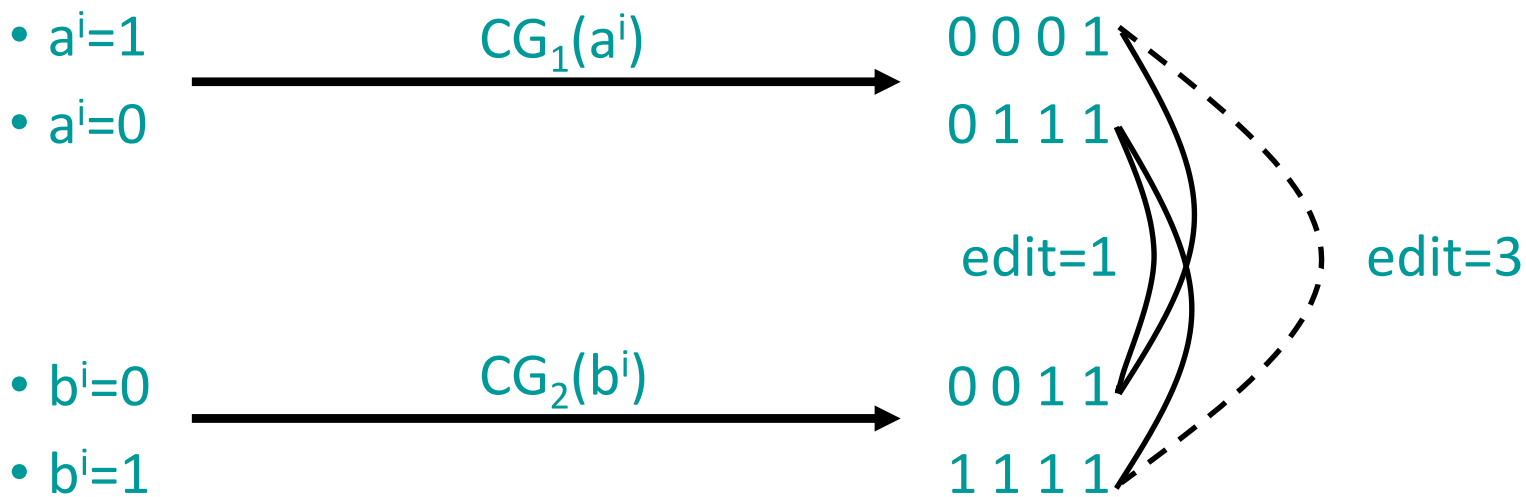
$$\alpha_1 \$ \alpha_2 \$ \alpha_3 \dots \alpha_{n-1} \$ \alpha_n$$

$$\underbrace{t \$ t \$ t \dots t \$ t}_{n-1 \text{ times}} \$ \beta_1 \$ \beta_2 \$ \beta_3 \dots \beta_{n-1} \$ \beta_n \$ \underbrace{t \$ t \$ t \dots t \$ t}_{n-1 \text{ times}}$$



Coordinate gadgets

- If $a^i \cdot b^i = 1$ then edit=large
- If $a^i \cdot b^i = 0$ then edit=small





Vector gadgets: first attempt

$\alpha' =$

- $a=a^1a^2\ldots a^d \rightarrow CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$

($22\ldots2$ has $d^{O(1)}$ twos)

$\beta' =$

- $b=b^1b^2\ldots b^d \rightarrow CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$

- edit(α',β')=?



Vector gadgets: first attempt

$\alpha' =$ $d^{O(1)}$

- $a=a^1a^2\ldots a^d \rightarrow CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$

$\beta' =$

- $b=b^1b^2\ldots b^d \rightarrow CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$

- $edit(\alpha',\beta')=d+2\cdot(\sum_i a^i\cdot b^i)$
- Bad attempt

What we want:

$edit(\alpha',\beta')=S$ when $a\cdot b=0$

$edit(\alpha',\beta')=L$ when $a\cdot b\neq0$



Vector gadgets

- $a=a^1a^2\ldots a^d \rightarrow \alpha' = CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$
- $b=b^1b^2\ldots b^d \rightarrow \beta' = CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$
- $\alpha'' = CG_1(0)\ 52\ldots2\ CG_1(0)\ \ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$

- $edit(\alpha',\beta')=d+2\cdot(\sum_i a^i\cdot b^i)$
- $edit(\alpha'',\beta')=?$



Vector gadgets

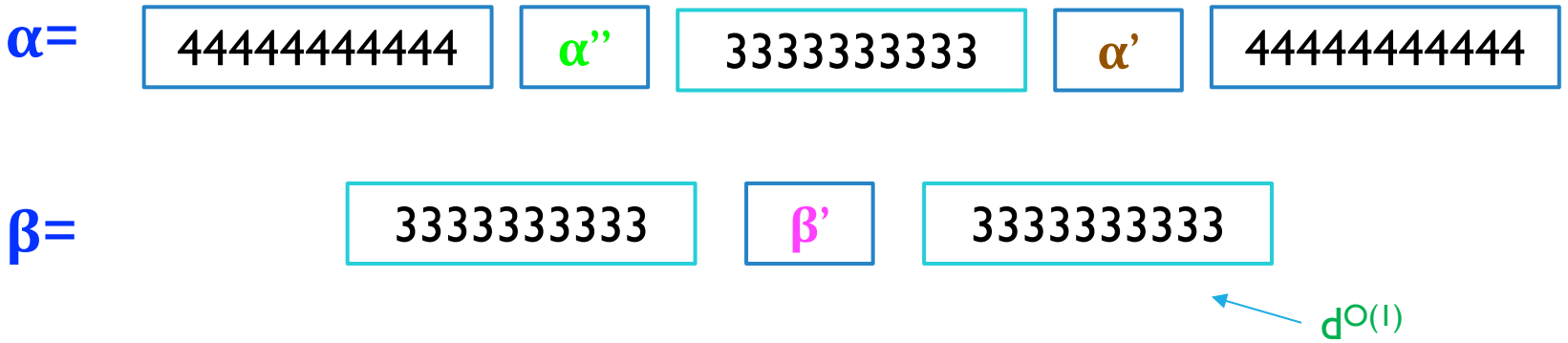
- $a=a^1a^2\ldots a^d \rightarrow \alpha' = CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$
- $b=b^1b^2\ldots b^d \rightarrow \beta' = CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$
- $\alpha'' = CG_1(0)\ 52\ldots2\ CG_1(0)\ \ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$

- $edit(\alpha'', \beta') = edit(\ CG_1(0)\ 52\ldots2\ CG_1(0)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$
  $CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)\ ) = d+1$
- $edit(\alpha', \beta') = d$ if $\Sigma_i a^i b^i = 0$
- $edit(\alpha', \beta') = d+2\Sigma_i a^i b^i \geq d+2$ if $\Sigma_i a^i b^i > 0$



Vector gadgets: final construction

- $a=a^1a^2\ldots a^d \rightarrow \alpha' = CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$
- $b=b^1b^2\ldots b^d \rightarrow \beta' = CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$
- $\alpha'' = CG_1(0)\ 52\ldots2\ CG_1(0)\ \ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$

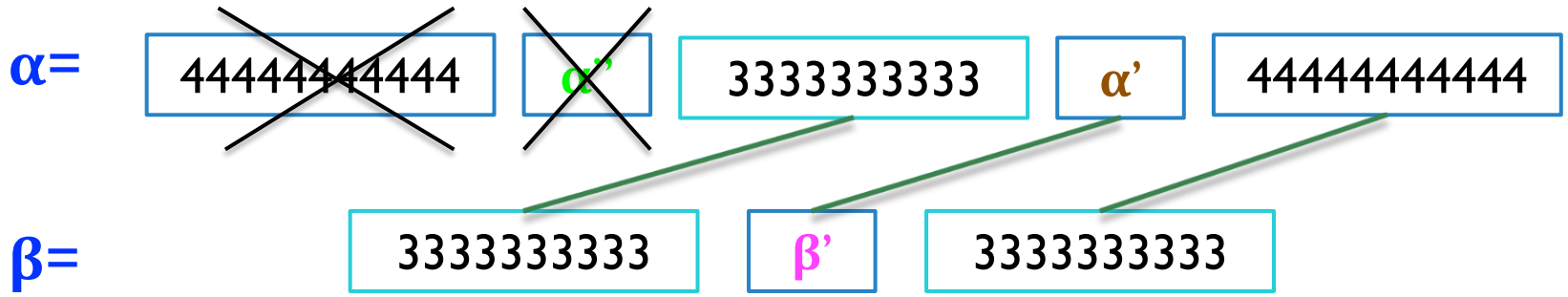


What is edit(α, β)?



Vector gadgets: final construction

- $a=a^1a^2\ldots a^d \rightarrow \alpha' = CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$
- $b=b^1b^2\ldots b^d \rightarrow \beta' = CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$
- $\alpha'' = CG_1(0)\ 52\ldots2\ CG_1(0)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$

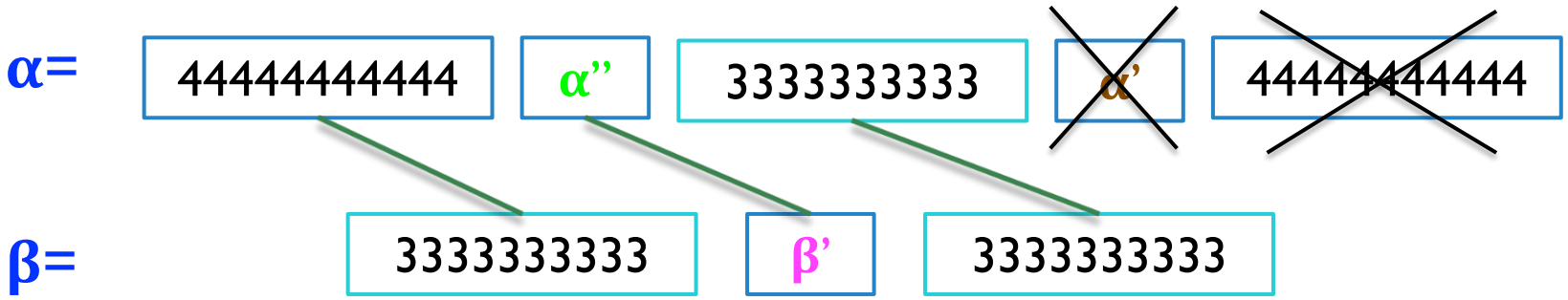


- Since $edit(\alpha',\beta')=d+2\cdot(\sum_i a^i\cdot b^i)$, $edit(\alpha'',\beta')=d+1$
- When $\sum_i a^i\cdot b^i=0$, $edit(\alpha',\beta')=d < edit(\alpha'',\beta')$



Vector gadgets: final construction

- $a=a^1a^2\ldots a^d \rightarrow \alpha' = CG_1(a^1)\ 22\ldots2\ CG_1(a^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_1(a^d)$
- $b=b^1b^2\ldots b^d \rightarrow \beta' = CG_2(b^1)\ 22\ldots2\ CG_2(b^2)\ 22\ldots2\ldots\ldots22\ldots2\ CG_2(b^d)$
- $\alpha'' = CG_1(0)\ 52\ldots2\ CG_1(0)\ \ 22\ldots2\ldots\ldots22\ldots2\ CG_1(0)$



- Since edit(α',β')=d+2·($\sum_i a^i \cdot b^i$), edit(α'',β')=d+1
- When $\sum_i a^i \cdot b^i > 0$, edit(α',β')≥d+2 > edit(α'',β')

What we get?

- Given $a=(a^1,\ldots,a^d)\in A$, $b=(b^1,\ldots,b^d)\in B$
- transform to strings α, β (of lengths $d^{O(1)}$), so that:

  if $a\cdot b=0$, edit$(\alpha, \beta)=S$

  if $a\cdot b>0$, edit$(\alpha, \beta)=S+1$



- Given $a=(a^1,\ldots,a^d)\in A$, $b=(b^1,\ldots,b^d)\in B$,
- transform to strings α, β (of lengths $d^{O(1)}$), so that:

  if $a\cdot b=0$, edit$(\alpha, \beta)=S$

  if $a\cdot b>0$, edit$(\alpha, \beta)=S+1$

- then for $A=\{a_1,\ldots,a_n\}$, $B=\{b_1,\ldots,b_n\}$,
  - transform to $\{\alpha_1,\ldots, \alpha_n\}, \{\beta_1,\ldots, \beta_n\}$
  - t is the strings transformed from vector $(1,\ldots,1)$
    - t is not orthogonal to any non-zero vector

$d^{O(1)}$

$$\alpha_1 \$ \alpha_2 \$ \alpha_3 \ldots \alpha_{n-1} \$ \alpha_n$$

$$\underbrace{t \$ t \$ t \ldots t \$ t}_{n-1 \text{ times}} \$ \beta_1 \$ \beta_2 \$ \beta_3 \ldots \beta_{n-1} \$ \beta_n \$ \underbrace{t \$ t \$ t \ldots t \$ t}_{n-1 \text{ times}}$$



- Given $a=(a^1,\ldots,a^d)\in A$, $b=(b^1,\ldots,b^d)\in B$,
- transform to strings α, β (of lengths $d^{O(1)}$), so that:

  if $a\cdot b=0$, edit$(\alpha, \beta)=S$

  if $a\cdot b>0$, edit$(\alpha, \beta)=S+1$

If there doesn't exist $a\in A$, $b\in B$ such that $a\cdot b=0$, then edit distance $\geq n(S+1)+Z$

- then for $A=\{a_1,\ldots,a_n\}$, $B=\{b_1,\ldots,b_n\}$,
  - transform to $\{\alpha_1,\ldots,\alpha_n\}, \{\beta_1,\ldots,\beta_n\}$
  - t is the strings transformed from vector $(1,\ldots,1)$
    - t is not orthogonal to any non-zero vector

$d^{O(1)}$

$$\alpha_1 \$ \alpha_2 \$ \alpha_3 \ldots \alpha_{n-1} \$ \alpha_n$$

$$\underbrace{t \$ t \$ t \ldots t \$ t}_{n-1 \text{ times}} \$ \beta_1 \$ \beta_2 \$ \beta_3 \ldots \beta_{n-1} \$ \beta_n \$ \underbrace{t \$ t \$ t \ldots t \$ t}_{n-1 \text{ times}}$$



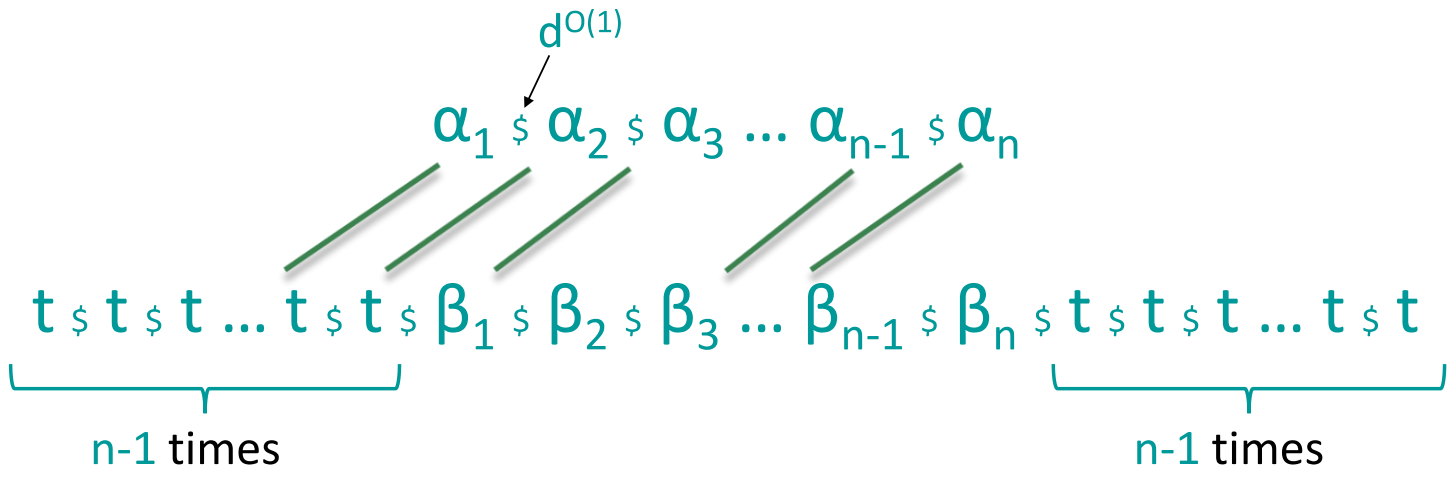
- Given $a=(a^1,\ldots,a^d)\in A$, $b=(b^1,\ldots,b^d)\in B$,
- transform to strings α, β (of lengths $d^{O(1)}$), so that:

  if $a\cdot b=0$, edit$(\alpha, \beta)=S$

  if $a\cdot b>0$, edit$(\alpha, \beta)=S+1$

If there exists $a\in A$, $b\in B$ such that $a\cdot b=0$,
(assume here it is a_3 and b_1)
then edit distance $\leq S+(n-1)(S+1)+Z$

- then for $A=\{a_1,\ldots,a_n\}$, $B=\{b_1,\ldots,b_n\}$,
  - transform to $\{\alpha_1,\ldots,\alpha_n\}, \{\beta_1,\ldots,\beta_n\}$
  - t is the strings transformed from vector $(1,\ldots,1)$
    - t is not orthogonal to any non-zero vector



OV $\Longrightarrow$ Edit Distance

- Given sets A, B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, we reduce to an edit distance instance of length $O(n \cdot d^{O(1)})$
- So if edit distance has an $O(n^{2-\delta})$ algorithm, we will have a $\tilde{O}(n^{2-\delta})$ algorithm for OV
  - full details in Backurs and Indyk's paper

- This shows edit distance is SETH-hard



Computing edit distance in $O(n^2/\log n)$ time?

- *[Masek, Paterson 1980]* Recall the dynamic programming

```
For each  i = 1…M
      For each  j = 1…N
                          { D(i-1,j) + 1
D(i,j)= min               { D(i,j-1) + 1
                          { D(i-1,j-1) +   1; if X(i) ≠ Y(j)
                                           0; if X(i) = Y(j)
```

	ϵ	G	C	T	A	T	G	C	C	A	C	G	C
ϵ	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2



Computing edit distance in $O(n^2/\log n)$ time?

- *[Masek, Paterson 1980]* Recall the dynamic programming
- if we look at a small $k \times k$ window on the table, it depends on:

	ϵ	G	C	T	A	T	G	C	C	A	C	G	C
ϵ	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2



Computing edit distance in $O(n^2/\log n)$ time?

- *[Masek, Paterson 1980]* Recall the dynamic programming
- if we look at a small $k \times k$ window on the table, it depends on:
  - The entries at the top and left of this window
  - The corresponding substrings

	ϵ	G	C	T	A	T	G	C	C	A	C	G	C
ϵ	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

So the k^2 elements in this $k \times k$ window depends only on O(k) elements



Computing edit distance in $O(n^2/\log n)$ time?

- Since two adjacent elements can only differ by -1, 0 or 1, the possible number of those "border entries" (relative to the upper-left corner) is only 3^{2k}
- Suppose the size of alphabet is 4, then the number of possible $k \times k$ window is $3^{2k} \times 4^{2k} = c^{2k}$ for some constant c
  - When $k=\Theta(\log n)$, there are $O(n)$ possible small windows, so we first list all of them

	ϵ	G	C	T	A	T	G	C	C	A	C	G	C
ϵ	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2



Computing edit distance in $O(n^2/\log n)$ time?

- The number of possible $k \times k$ window is c^{2k} for some constant c
  - When $k=\Theta(\log n)$, there are O(n) possible small windows, so we first list all of them
- Partition the table into $k \times k$ small tables, it will be like we only compute the border entries for each small table
- Time: $(n/k) \times (n/k)$ small tables, with O(k) time for each small table
  - $O(n^2/k)=O(n^2/\log n)$

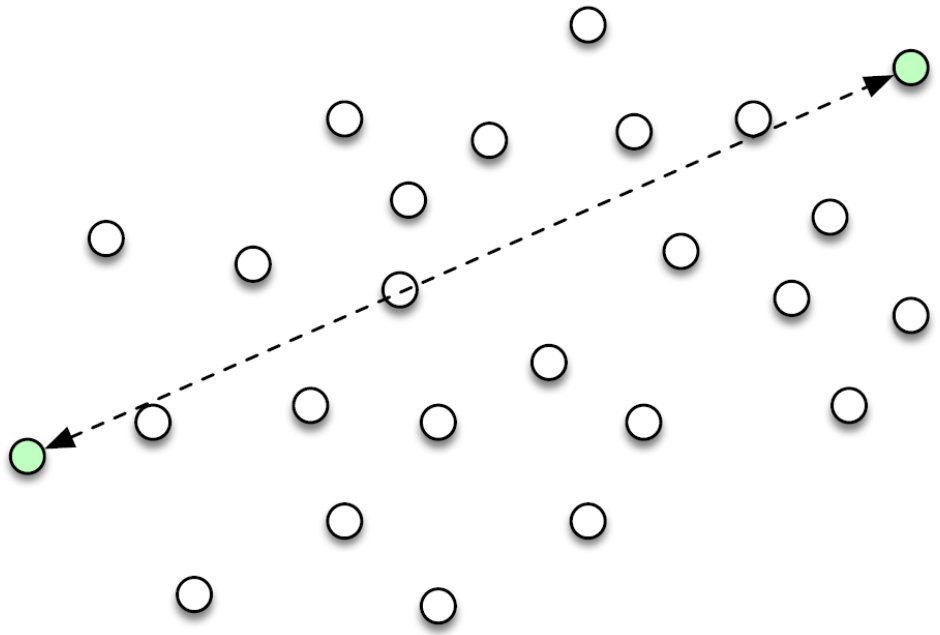
	ϵ	G	C	T	A	T	G	C	C	A	C	G	C
ϵ	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2



Other SETH-hard problems

Diameter

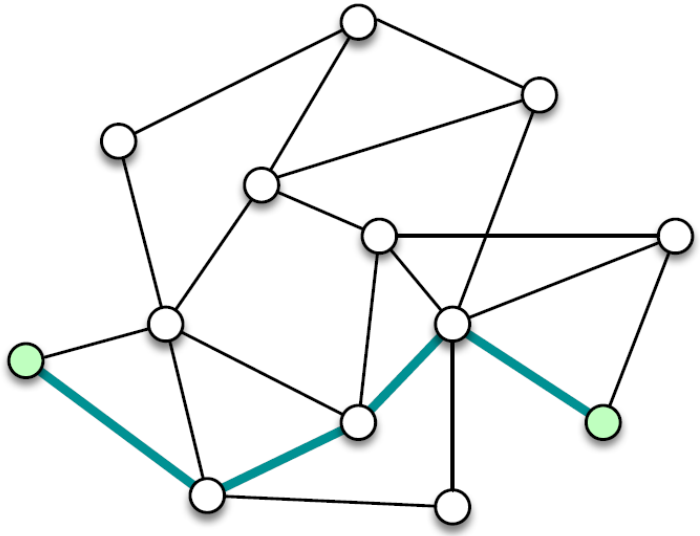
- Let d(u,v) be the distance from u to v in the graph G
- The diameter D of G is $\max_{u,v \in G} d(u,v)$
  - "the largest distance between its vertices"





Diameter

- In an undirected unweighted graph G, we want to find its diameter D
- Trivial way: finding all-pair distances
  - BFS from all vertices, time: O(mn)





Approximation Algorithms

- sometimes happy with approximately optimal solution
  - we want approximation algorithm with guaranteed approximation ratio of r>1
  - meaning: on every input x, output is guaranteed to have value

    ≤r*opt for minimization

    ≥opt/r for maximization



3/2-approximation of the diameter

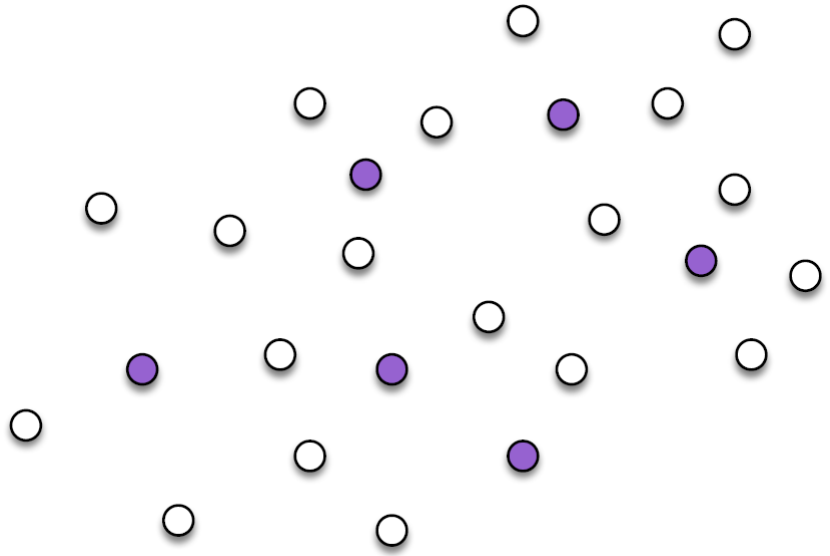
- *[Roditty, V. Williams 2014]* The diameter of unweighted graphs can be 3/2-approximated in $\tilde{O}(mn^{1/2})$ time
  - output an estimate $\check{D}$ such that $\lfloor 2/3 \cdot D \rfloor \leq \check{D} \leq D$ (D is the real diameter)

- Here we assume:
  - D=3h for some integer h
  - work on undirected graphs



3/2-approximation of the diameter

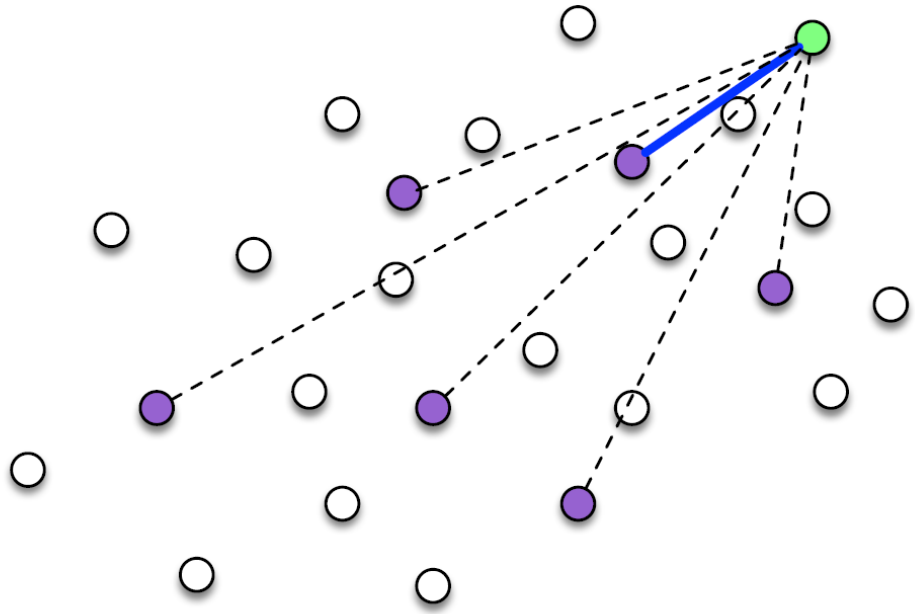
- First we randomly sample a vertex set S by including every vertex with probability $1/\sqrt{n}$ independently
- What's the size of S?
  - expected size: $O(\sqrt{n})$
  - with prob. 1-1/n^c, $|S| = O(\sqrt{n}\log n) = \tilde{O}(\sqrt{n})$





3/2-approximation of the diameter

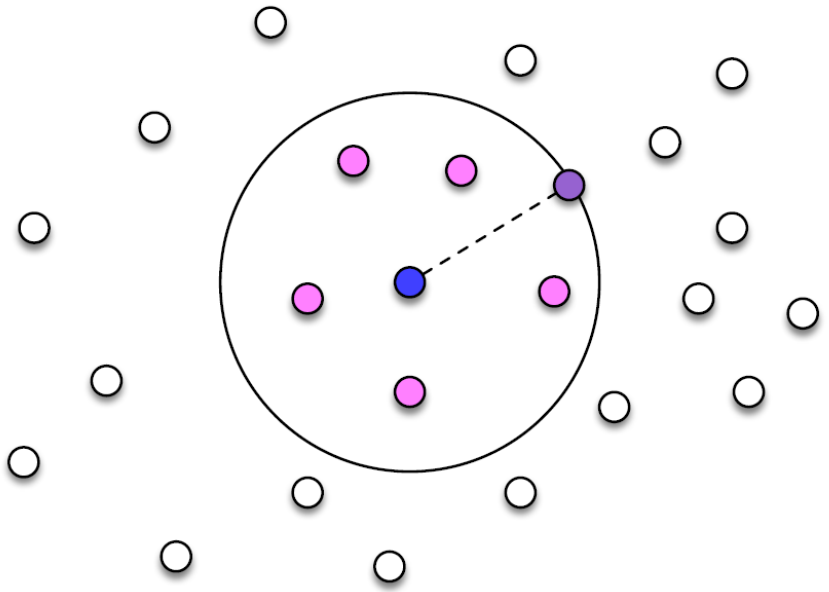
- For a vertex u, define d(u,S) to be $\min_{v \in S} d(u,v)$
- Compute d(u,S) for every u
  - BFS from all vertices of S
  - Time: $\tilde{O}(m\sqrt{n})$





3/2-approximation of the diameter

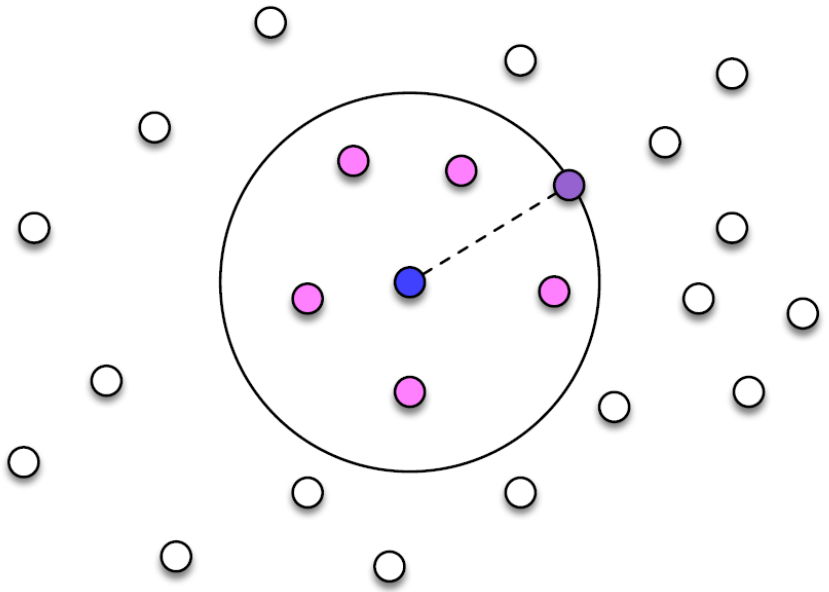
- Find the vertex w that maximizes d(w,S)
- Find the ball T={v: d(w,v)<d(w,S)}
  - What's the size of T?
  - w.h.p. $\tilde{O}(\sqrt{n})$





3/2-approximation of the diameter

- Find the vertex w that maximizes d(w,S)
- Find the ball T={v: d(w,v)<d(w,S)}
- Run BFS from vertices in S and T, and return the longest distance
  - Time: $\tilde{O}(m\sqrt{n})$
  - approximation ratio?

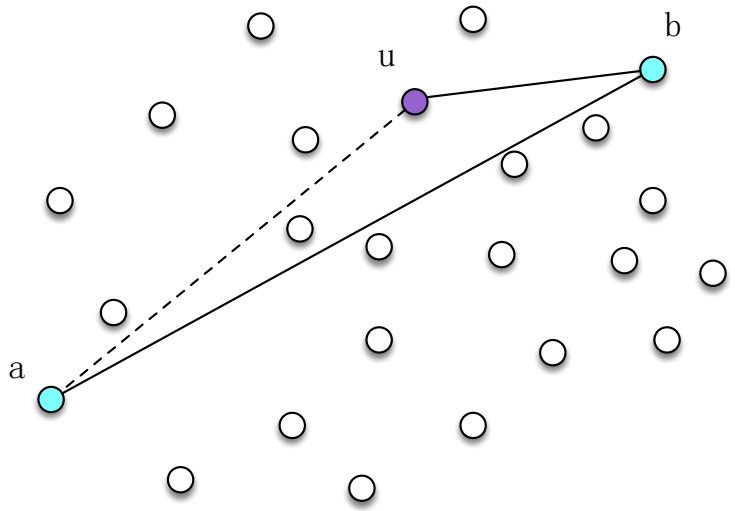




The algorithm

- Randomly sample a vertex set S by including every vertex with probability $1/\sqrt{n}$ independently
- Compute d(u,S) for every u
- Find the vertex w that maximizes d(w,S), and find the ball T={v: d(w,v)<d(w,S)}
- Run BFS from vertices in S and T, and return the longest distance

- Consider the real diameter D=d(a,b)=3h
- If there is a vertex u∈S such that either:
  - d(u,a)≥2h or d(u,b)≥2h
  - d(u,a)≤h or d(u,b)≤h (*since d(u,b)≥d(a,b)-d(u,a)*)
- the BFS from u can get a 3/2-approximation

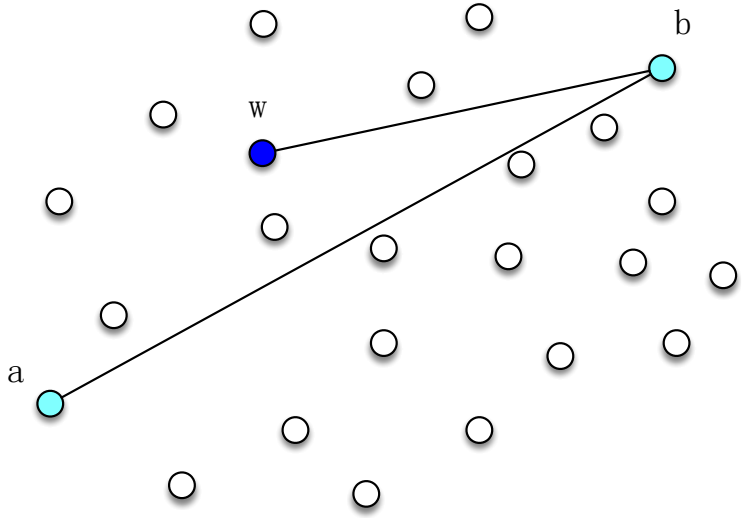




The algorithm

- Randomly sample a vertex set S by including every vertex with probability $1/\sqrt{n}$ independently.
- Compute d(u,S) for every u
- Find the vertex w that maximizes d(w,S), and find the ball T={v: d(w,v)<d(w,S)}
- Run BFS from vertices in S and T, and return the longest distance.

- What's left is, for all u∈S,
  - h<d(u,a)<2h and h<d(u,b)<2h
- This means d(a, S)>h
- So d(w, S)>h
- Since w∈T, if d(w,b)≥2h, we can also find a 3/2-approximation by BFS
- So we only consider d(w,b)<2h

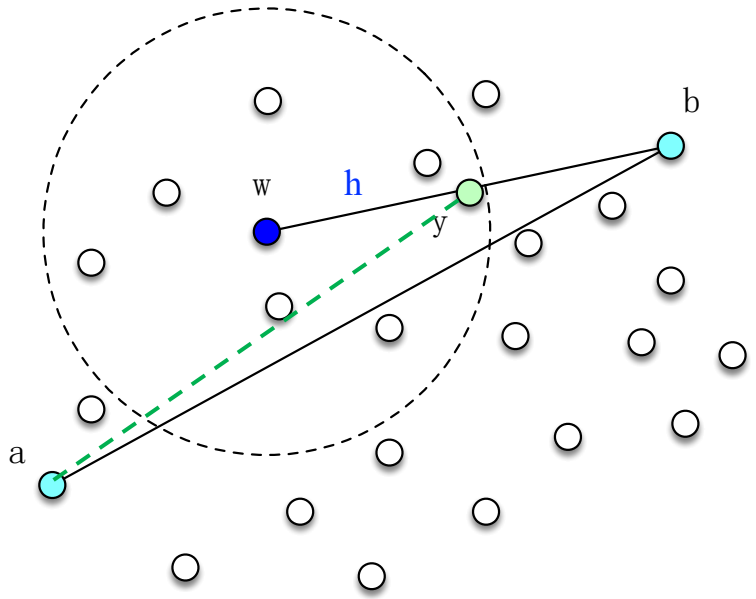




The algorithm

- Randomly sample a vertex set S by including every vertex with probability $1/\sqrt{n}$ independently.
- Compute d(u,S) for every u
- Find the vertex w that maximizes d(w,S), and find the ball T={v: d(w,v)<d(w,S)}
- Run BFS from vertices in S and T, and return the longest distance.

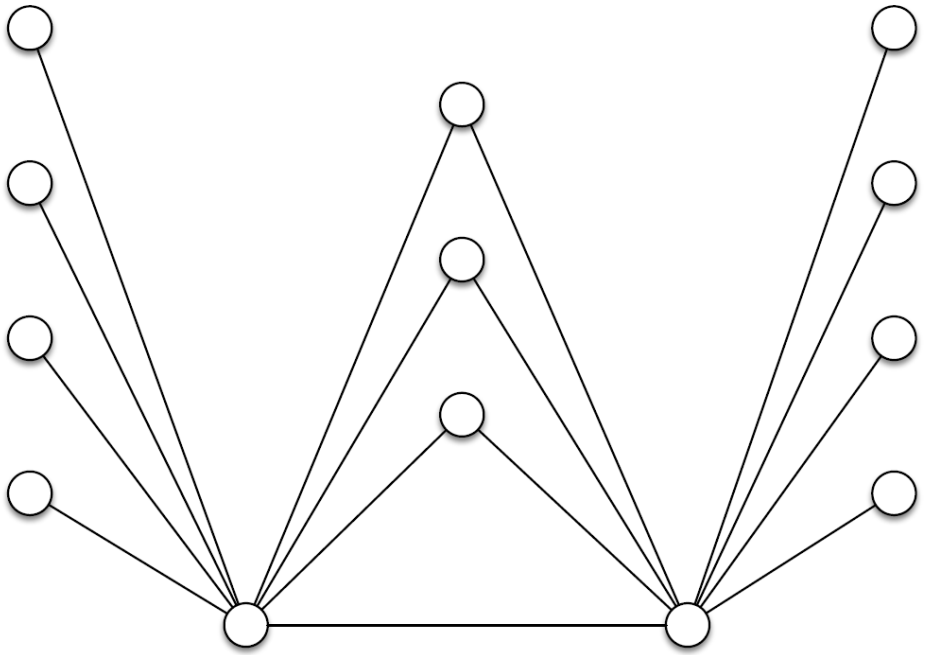
- What's left is, for all u∈S,
  - h<d(u,a)<2h and h<d(u,b)<2h
- So we only consider d(w,b)<2h
- Since d(w,S)>h, the vertex y on the path from w to b with distance h from w must be in T
- Since d(w,y)=h, d(y,b)<h, so d(a,y)>2h
- We can get a 3/2-approximation by BFS from y∈T





Hardness of diameter

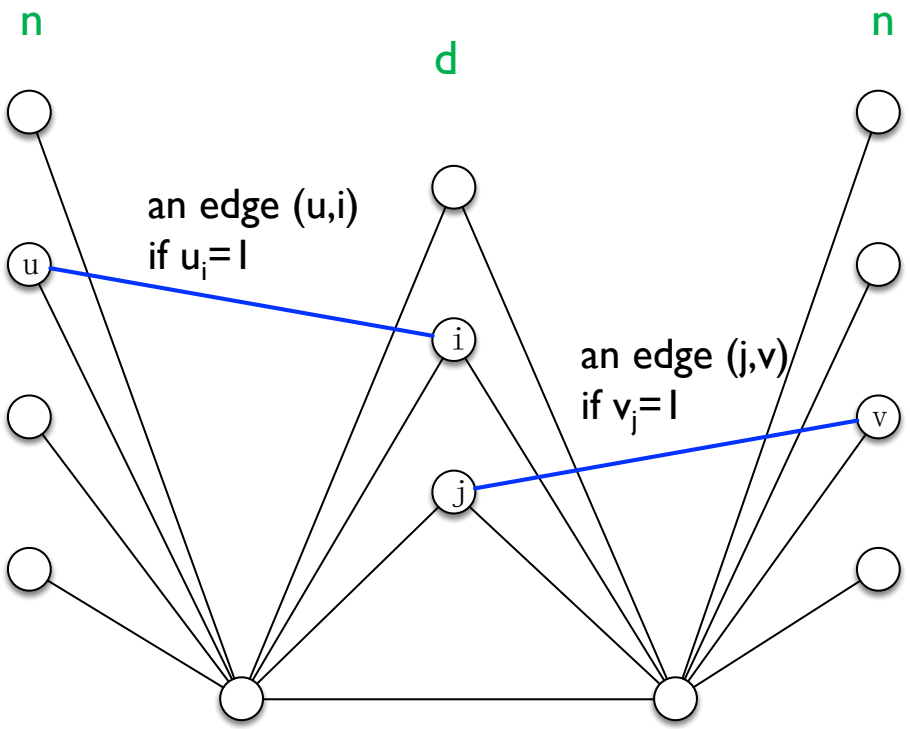
- A simple reduction from OV to diameter
  - No $O(m^{2-\delta})$ time algorithm can compute the diameter better than 3/2-approximation under OV conjecture *[Roditty, V. Williams 2014]*





Hardness of diameter

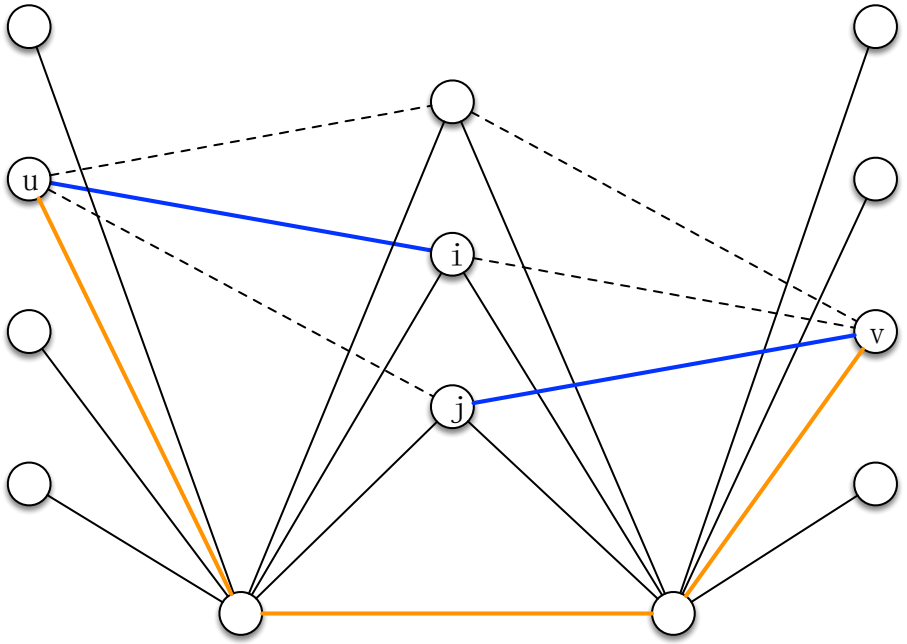
- A simple reduction from OV to diameter
  - $m=\tilde{O}(n)$





Hardness of diameter

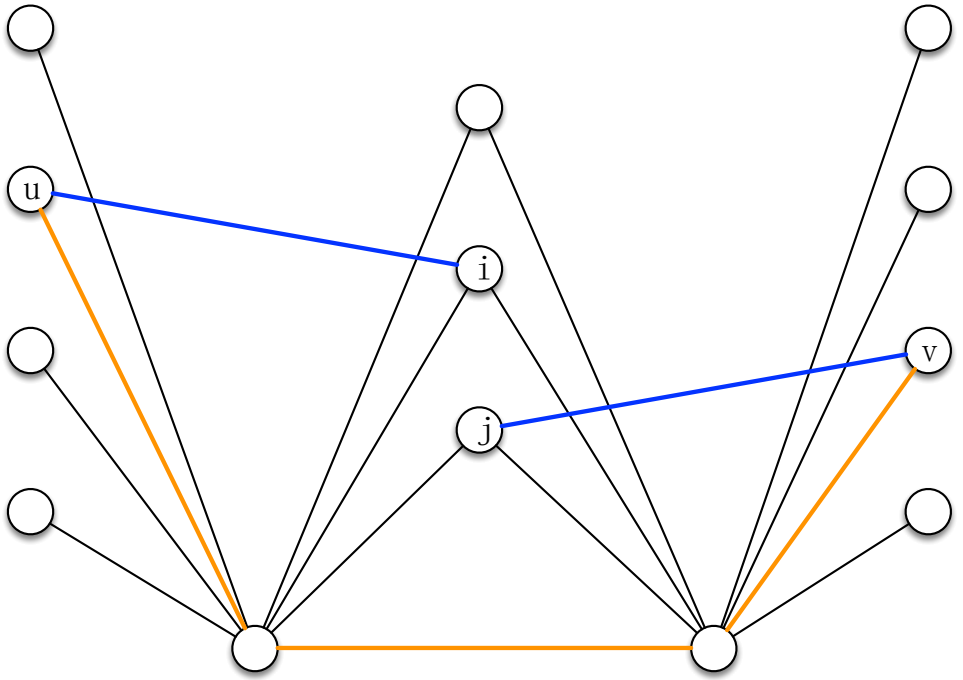
- A simple reduction from OV to diameter
  - if (u,v) are orthogonal, their distance is 3
  - if not, their distance is 2





Hardness of diameter

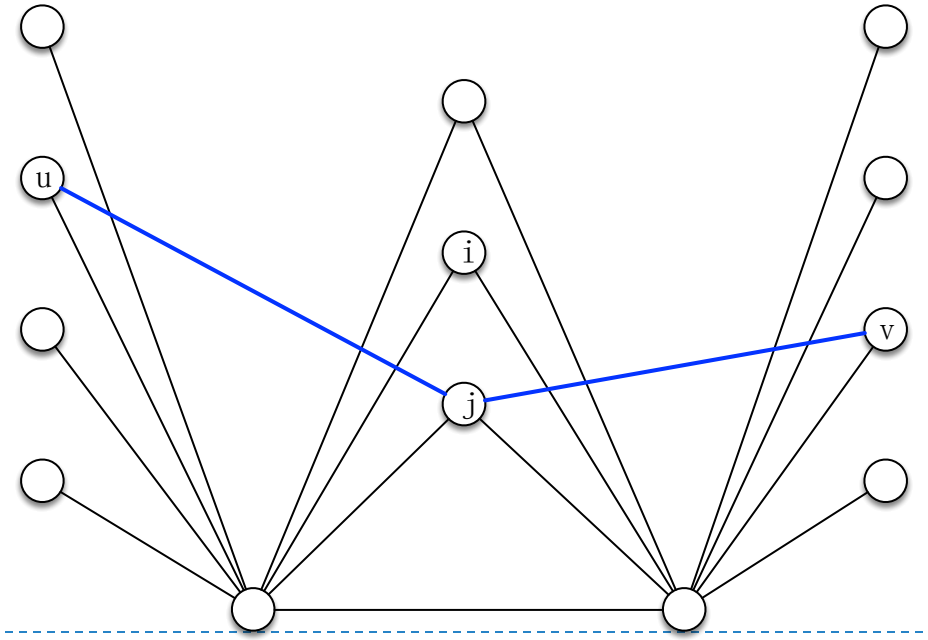
- A simple reduction from OV to diameter
  - diameter=3 if there is a pair of orthogonal vectors (u,v)





Hardness of diameter

- A simple reduction from OV to diameter
  - diameter=3 if there is a pair of orthogonal vectors (u,v)
  - diameter=2 if there is no pair of orthogonal vectors
  - Thus, if we can distinguish 2 or 3 for diameter, we can solve OV





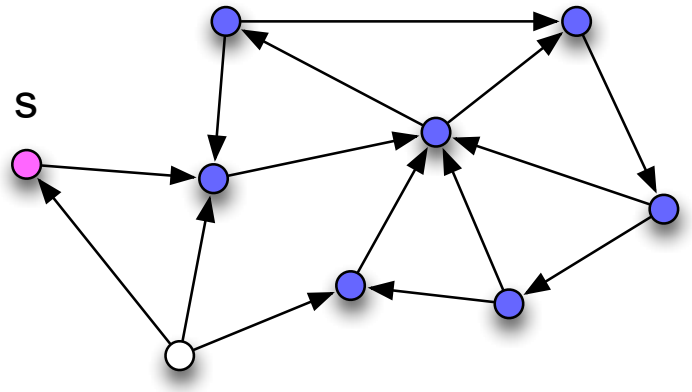
Summary *[Roditty, V. Williams 2014]*

- The diameter of unweighted graphs can be 3/2-approximated in $\tilde{O}(mn^{1/2})$ time
  - output an estimate $\check{D}$ such that $\lfloor 2/3 \cdot D \rfloor \leq \check{D} \leq D$
- No $O(m^{2-\delta})$ time algorithm can compute the diameter better than 3/2-approximation under OV conjecture
- Still no $O(n^{3-\delta})$ time algorithm for computing diameter, also no proof that it is APSP-hard



Single Source Reachability

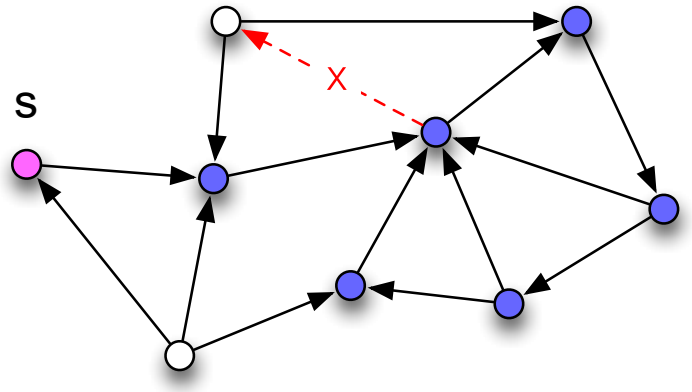
- Input: a directed graph G, a source node s
- Updates: Add or remove edges
- Query: #SSR How many nodes can s reach?





Single Source Reachability

- Input: a directed graph G, a source node s
- Updates: Add or remove edges
- Query: #SSR How many nodes can s reach?





Single Source Reachability

- Input: a directed graph G, a source node s
- Updates: Add or remove edges
- Query: #*SSR* How many nodes can s reach?

- Trivial: O(m) time depth-first search
- is there $O(m^{1-\delta})$ update/query time algorithm?



What about undirected graph?

- Dynamic connectivity:
  - We have lots of truly sublinear update time algorithm

    - (Deterministic, Amortized) : $O(\log^2 n)$ [HDLT01].
    - (Randomized, Amortized) : $O(\log n(\log\log n)^2)$ [HHKP17].
    - (Randomized, Worst-Case, None-Adaptive) : $O(\log^4 n)$ [KKM13, Wan15, GKKT15].
    - (Randomized, Worst-Case, Adaptive, Monte-Carlo) : $O(n^{0.4+o(1)})$ [NS16].
    - (Randomized, Worst-Case, Adaptive, Las-Vegas) : $O(n^{0.49305+o(1)})$ [NS16]
    - (Deterministic, Worst-Case): $O\left(\sqrt{\frac{n(\log\log n)^2}{\log n}}\right)$ [KRKPT15].



SETH-hard

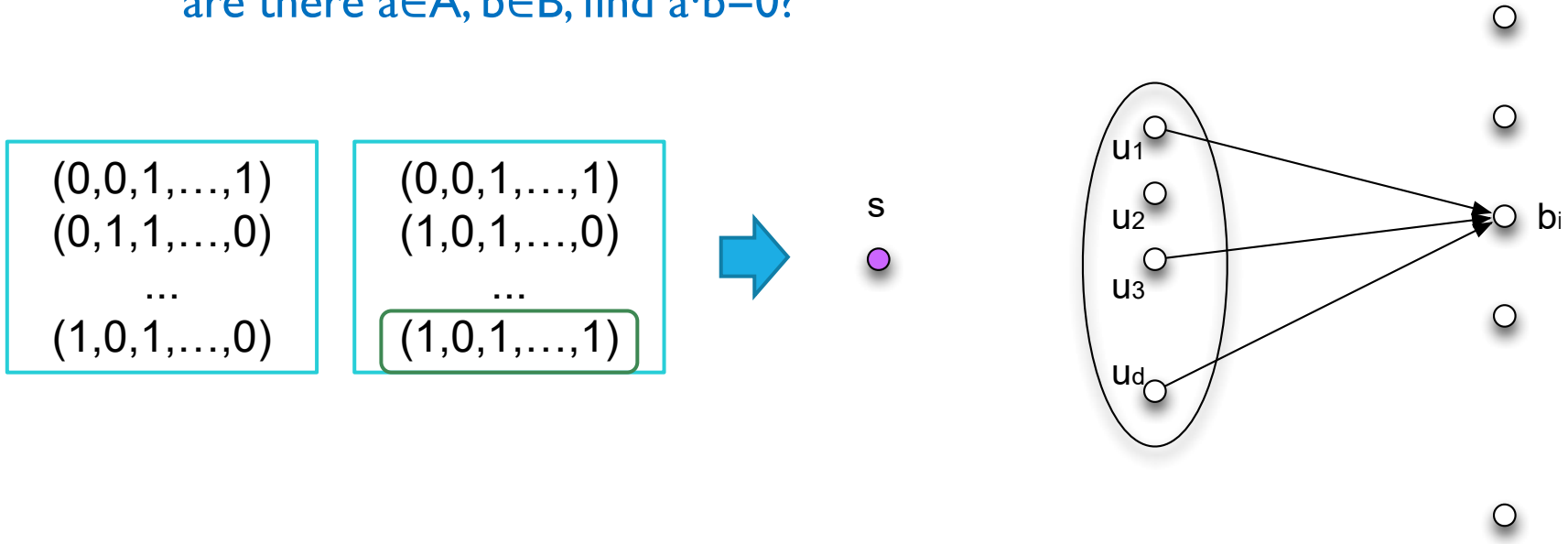
- Input: a directed graph G, a source node s
- Updates: Add or remove edges
- Query: *#SSR* How many nodes can s reach?

- *[Abbound, V. Williams 2014]* if dynamic #SSR can be solved in $O(m^{1-\delta})$ time (both update and query time), then orthogonal vector can be solved in truly subquadratic time, (then SETH fails)



OV ⟹ Dynamic #SSR

OV: Given sets A,B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a \in A$, $b \in B$, find $a \cdot b = 0$?

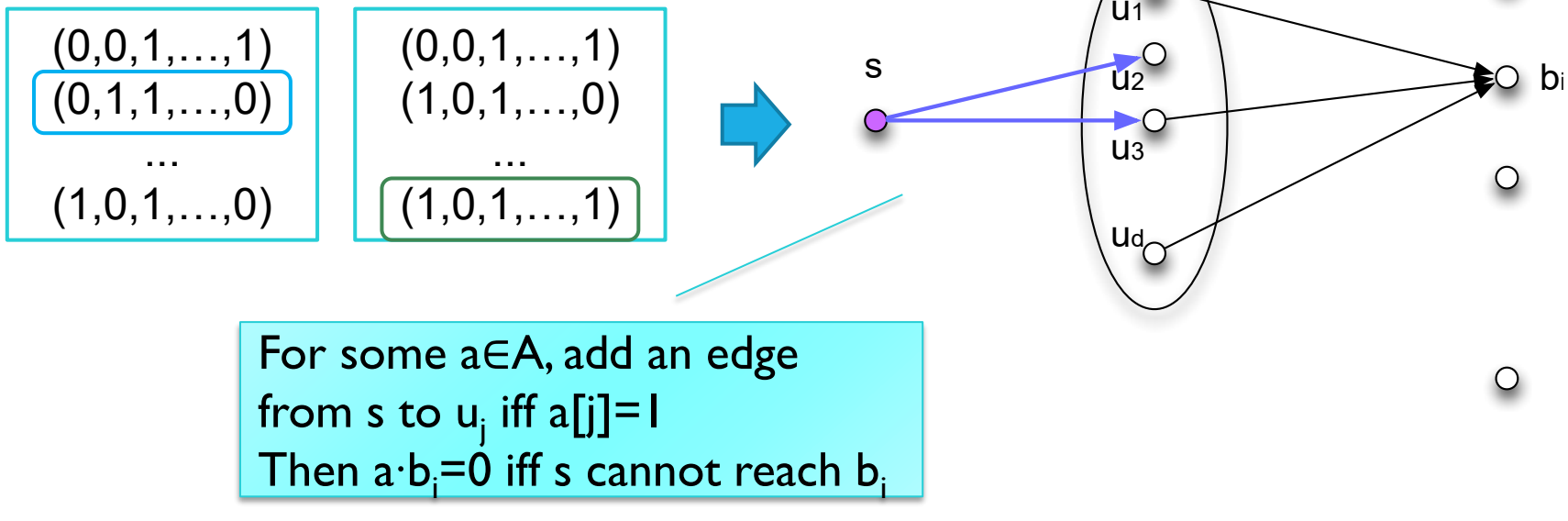


For every $b_i \in B$, add an edge from u_j to b_i iff $b_i[j]=1$



OV ⟹ Dynamic #SSR

OV: Given sets A,B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a \in A$, $b \in B$, find $a \cdot b = 0$?



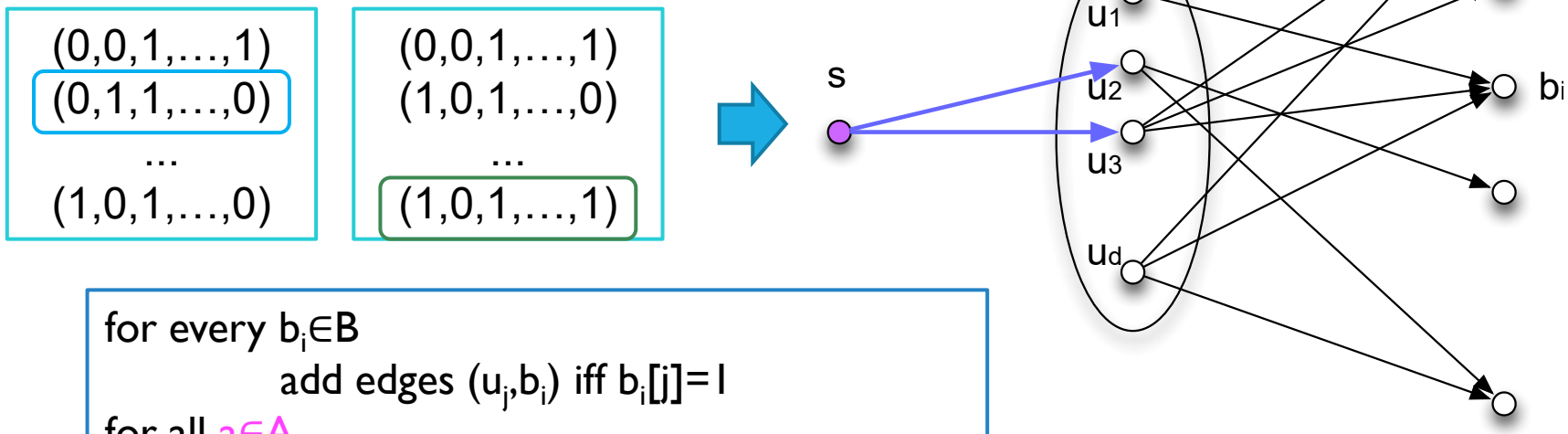
For some $a \in A$, add an edge from s to u_j iff $a[j]=1$
Then $a \cdot b_i = 0$ iff s cannot reach b_i

For every $b_i \in B$, add an edge from u_j to b_i iff $b_i[j]=1$



OV ⟹ Dynamic #SSR

OV: Given sets A,B of n vectors in $\{0,1\}^d$, for d=O(log n), are there $a \in A$, $b \in B$, find $a \cdot b = 0$?



for every $b_i \in B$
    add edges (u_j, b_i) iff $b_i[j]=1$
for all $a \in A$
    add edges (s, u_j) iff $a[j]=1$
    if #SSR(s)< n+ (1s in a)
        return YES
    remove all edges from s
return NO



OV $\Longrightarrow$ Dynamic #SSR

OV: Given sets A,B of n vectors in $\{0,1\}^d$, for $d=O(\log n)$, are there $a \in A$, $b \in B$, find $a \cdot b = 0$?

```
for every b_i∈B
        add edges (u_j,b_i) iff b_i[j]=1
for all a∈A
        add edges (s,u_j) iff a[j]=1
        if #SSR(s)< n+ (1s in a)
                return YES
        remove all edges from s
return NO
```

- O(nd) updates, the graph has O(nd) edges, O(n) queries
- if update and query time is $O(m^{1-\delta})$ for constant $\delta>0$, which will be $\tilde{O}(n^{1-\delta})$ here, then OV can be solved in $\tilde{O}(n^{2-\delta})$ time



Lower bounds with much better guarantees!

- *[Pătraşcu 2010]* Find $O(n^{o(1)})$ time update/query time algorithm for many dynamic problems is 3SUM-hard
  - dynamic reachability
  - subgraph connectivity
  - Langerman's problem
  - …

- *[Abboud, Vassilevska, Yu 2015]* Even if ***at least one*** of APSP, 3SUM, SETH is true, then Single Source Reachability requires linear time updates!

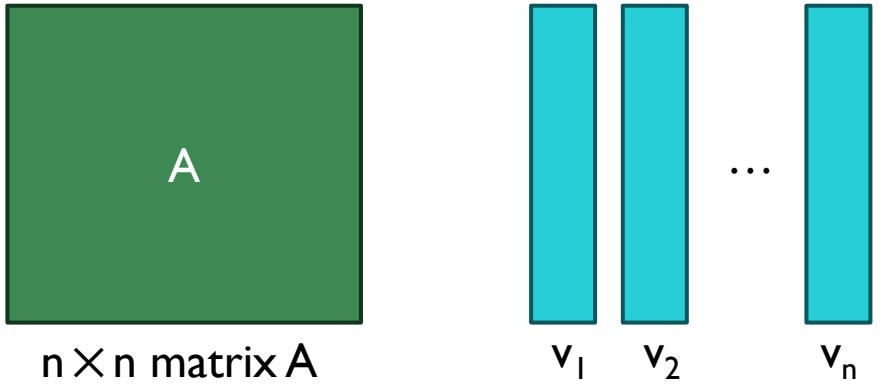


Online Matrix-Vector Multiplication

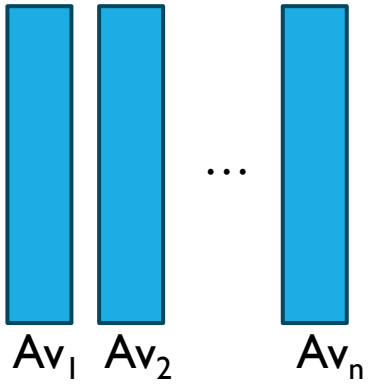
Online matrix-vector multiplication conjecture (OMv)

[Henzinger et al. '15]

INPUT:



OUTPUT:



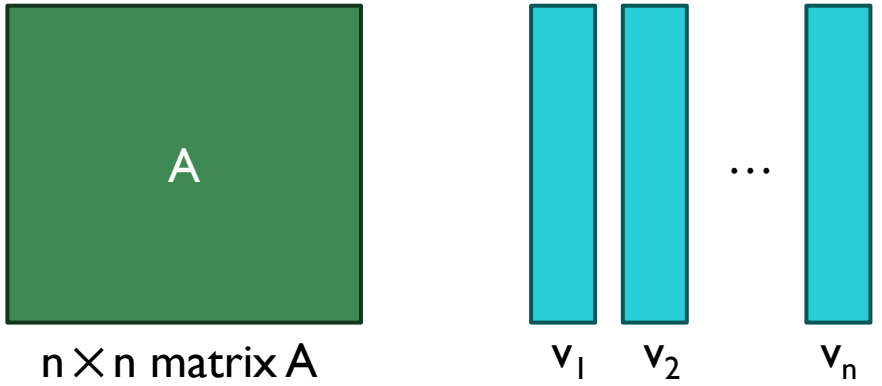
Online: you need to answer $A \cdot v_i$ before next v_{i+1} arrives



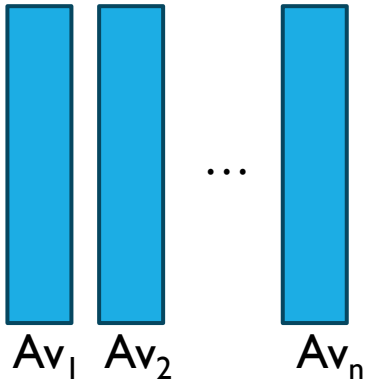
Online matrix-vector multiplication conjecture (OMv)

[Henzinger et al. '15]

INPUT:



OUTPUT:



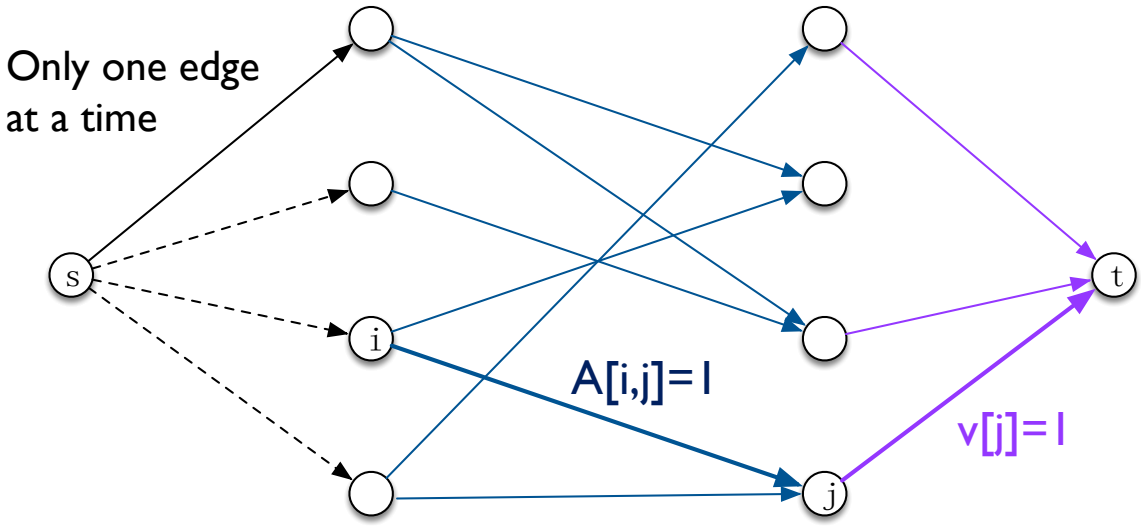
Online: you need to answer $A \cdot v_i$ before next v_{i+1} arrives

Offline: by matrix multiplication in $O(n^{2.3714})$ time

Conjecture: no algorithms for online version in $O(n^{3-\delta})$ time for $\delta>0$

st-Reachability in directed graphs

- Reduce OMv to dynamic st-reachability:



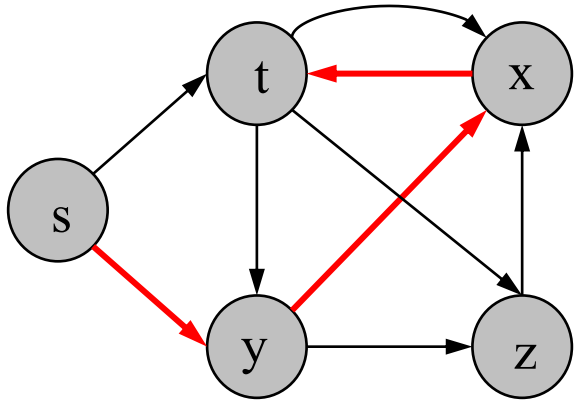
for every $A \cdot v$ we need to update n edges to t
for every $(A \cdot v)[i]$ we need to update one edge from s and one query
$O(n^2)$ updates and queries

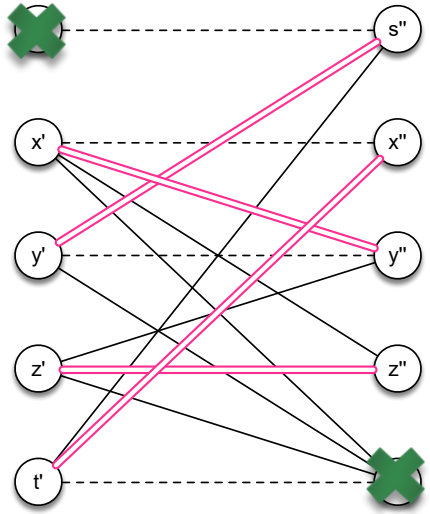
Dynamic st-reachability in $O(n^{0.99})$ update and query time will refute OMv conjecture



s-t reachability to matching

- duplicate all vertex v to v' and v"
- an edge (u,v) → (u",v')
- add (v',v")
- delete s' and t"

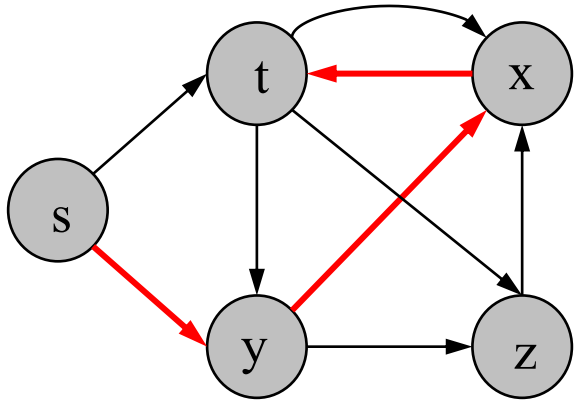


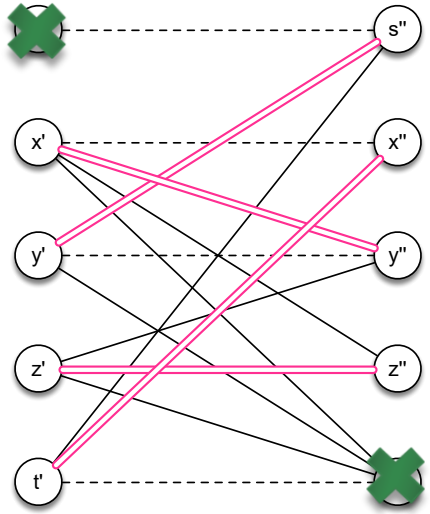




s-t reachability to matching

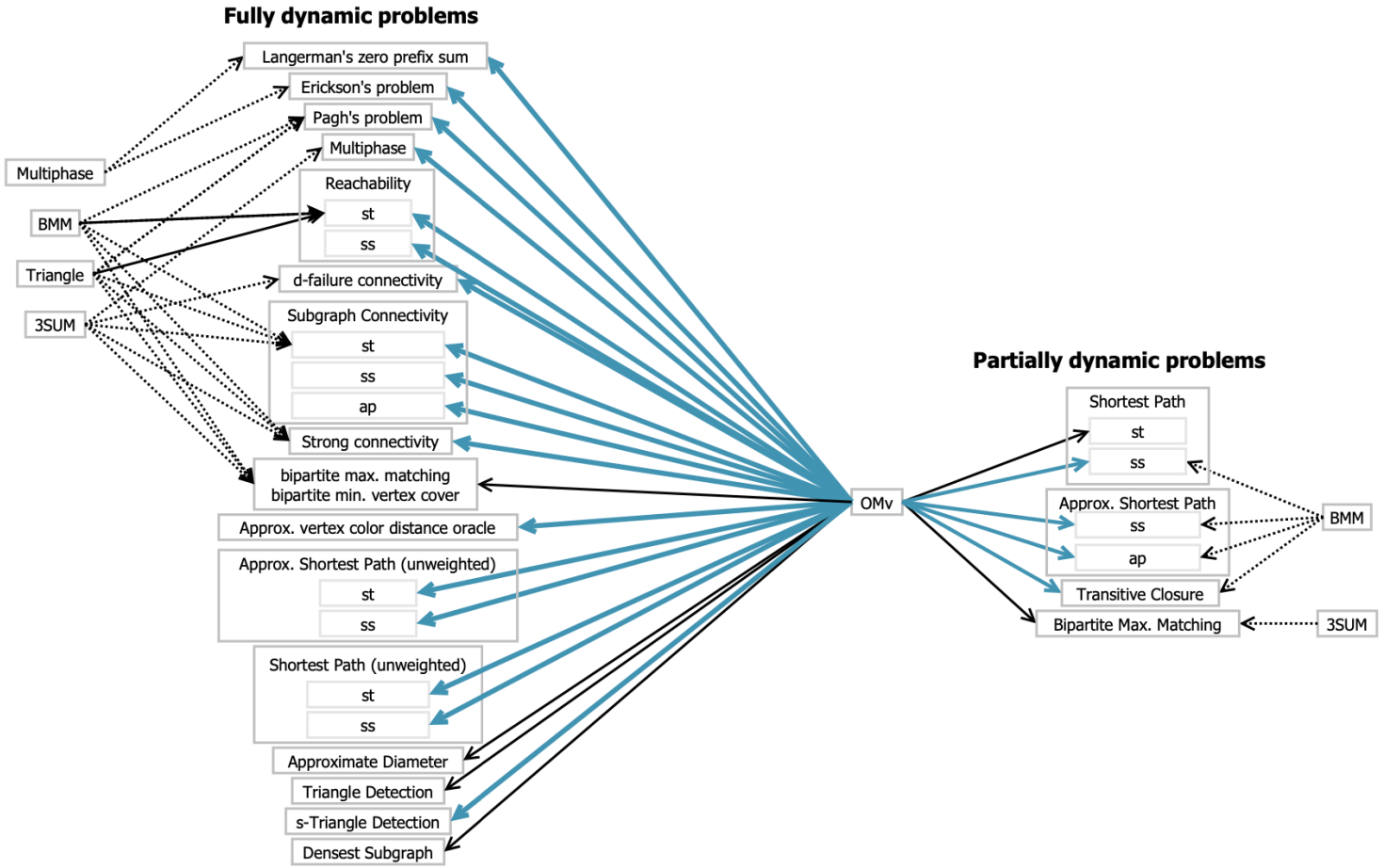
- Then a perfect matching in the new graph corresponds to an s-t path and some edges (v',v")







- s-t reachability ⇒ bipartite perfect matching
- Thus, dynamic BP-matching also does not have $O(n^{0.99})$ update and query time algorithm under OMv conjecture



[Henzinger, Krinninger, Nanongkai, Saranurak '15]

3SUM Hardness

3SUM hardness

- *[Gajentaan, Overmars 1995]* Many computational geometry problems are 3SUM-hard to get $O(n^{2-\delta})$ time algorithm
  - Given n points in plane, are there 3 in a line?
  - Given n lines in plane, are there 3 pass through one point?
  - Given n strips in the plane, does their union contain a given rectangle
  - Given n triangles in the plane, does their union contain another given triangle
  - …

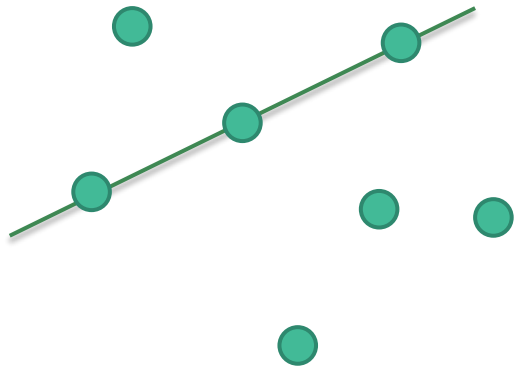
3SUM Conjecture: 3SUM on n integers in $\{-n^3,\ldots,n^3\}$ requires $n^{2-o(1)}$ time.



3-point collinear

- Given n points in the plane, are any three collinear?

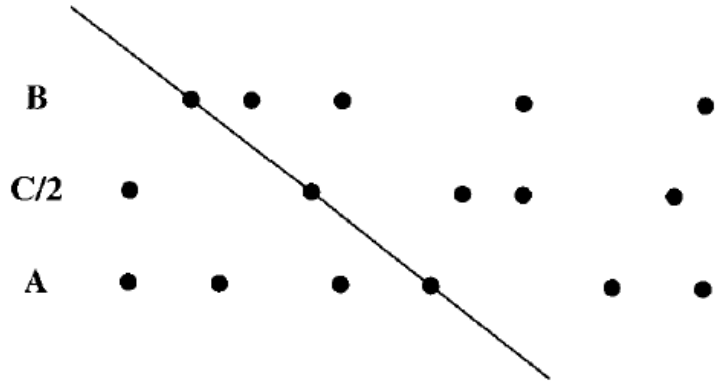
- Basic algorithm:
  - Draw the line between every pair
  - Time: $O(n^2)$ *[Edelsbrunne 89]*
  - still no $O(n^{2-\delta})$ time algorithm





3SUM $\Longrightarrow$ 3-point collinear

- Given a set S of integers, are there a,b,c∈S so that a+b+c=0?
- If we just want to find non-horizontal line, then it is trivial:
  - are there a∈A, b∈B, c∈C, such that a+b=c?

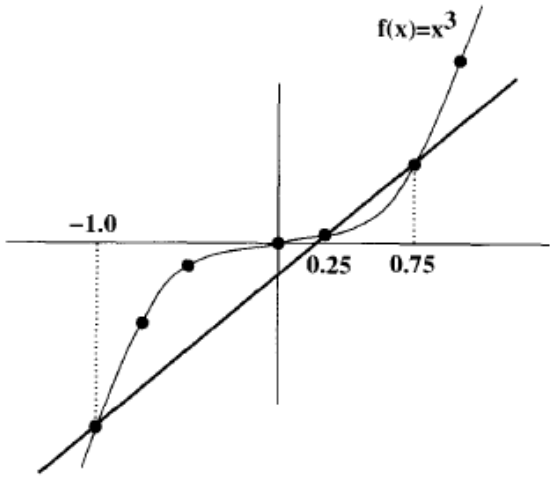


What about the general case?



3SUM $\Longrightarrow$ 3-point collinear

- Given a set S of integers, are there a,b,c∈S so that a+b+c=0?
- For every a∈S, create a point (a, a^3) in plane
- **Claim**: a+b+c=0 iff (a, a^3), (b, b^3), (c, c^3) are collinear





3SUM $\Longrightarrow$ Point on 3-line

- Given a set of lines in the plane, is there a point that lies on at least three of them?
  - it's dual to the 3-point on line



3SUM $\Longrightarrow$ Point on 3-line

- Given a set of lines in the plane, is there a point that lies on at least three of them?
  - it's dual to the 3-point on line

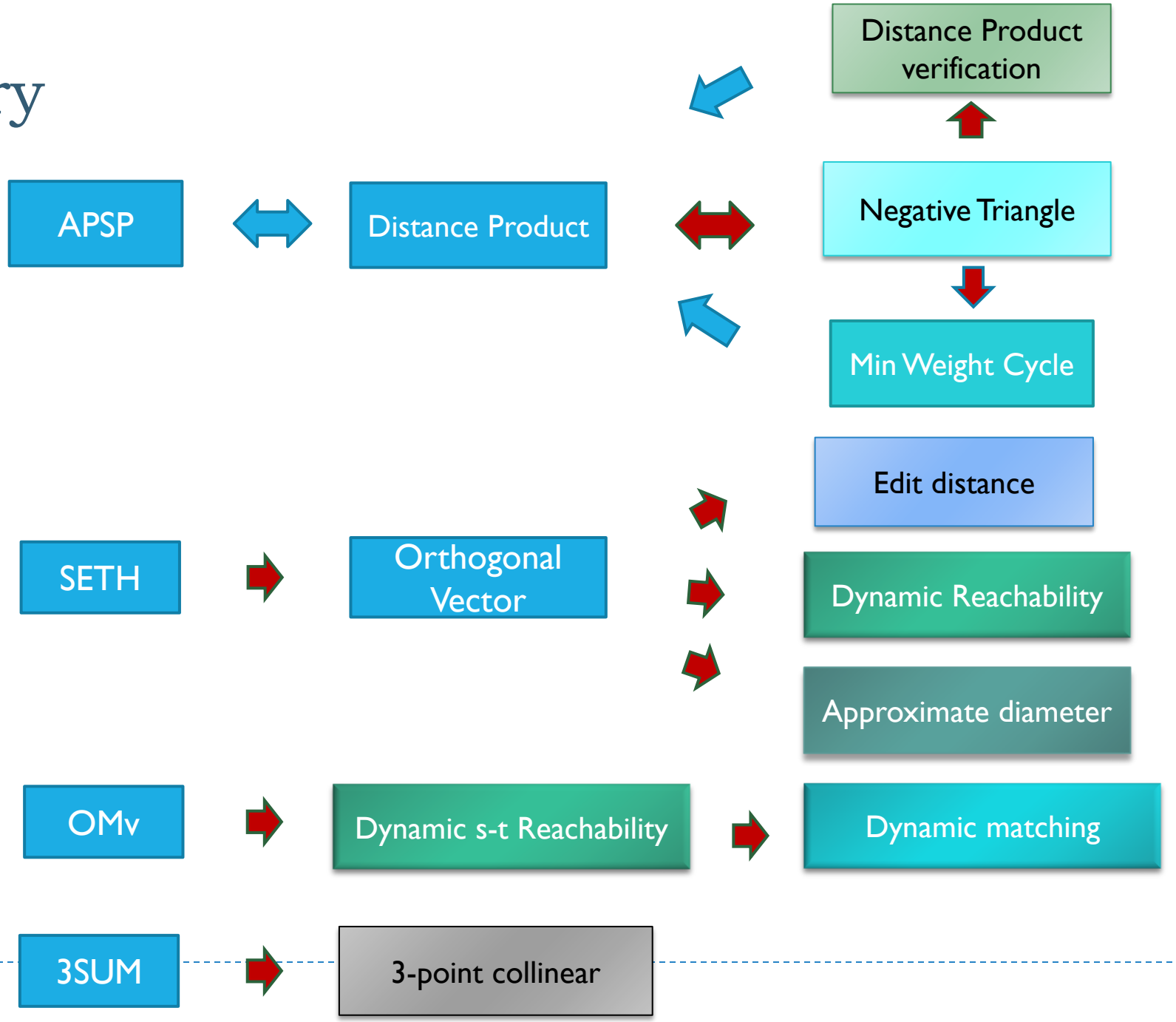
$$\begin{aligned} \mathrm{p} = (a, b) \quad &\Rightarrow \quad \mathrm{p}^\star : y = ax - b \\ \ell : y = cx + d \quad &\Rightarrow \quad \ell^\star = (c, -d). \end{aligned}$$

so p is on $\ell \Leftrightarrow$ ac+d=b $\Leftrightarrow$ $\ell^\star$ is on $\mathrm{p}^\star$

For further reductions see Gajentaan and Overmars's paper *"On a class of $O(n^2)$ problems in computational geometry"*



Summary


Distance Product verification
APSP
Distance Product
Negative Triangle
Min Weight Cycle
Edit distance
SETH
Orthogonal Vector
Dynamic Reachability
Approximate diameter
OMv
Dynamic s-t Reachability
Dynamic matching
3SUM
3-point collinear

- Final exam
  - June 9 in-class
  - Location:6A116
  - Fully closed-book

- Office hour
  - every Monday 4:00-5:00pm, FIT 4-6013

